

A Graph-Native Approach to Normalization

Johannes Schrott

TU Wien

Vienna, Austria

johannes.schrott@tuwien.ac.at

Maxime Jakubowski

TU Wien

Vienna, Austria

maxime.jakubowski@tuwien.ac.at

Katja Hose

TU Wien

Vienna, Austria

katja.hose@tuwien.ac.at

Abstract

In recent years, knowledge graphs (KGs) – in particular in the form of labeled property graphs (LPGs) – have become essential components in a broad range of applications. Although the absence of strict schemas for KGs facilitates structural issues that lead to redundancies and subsequently to inconsistencies and anomalies, the problem of KG quality has so far received only little attention. Inspired by normalization using functional dependencies for relational data, a first approach exploiting dependencies within nodes has been proposed. However, real-world KGs also expose functional dependencies involving edges. In this paper, we therefore propose graph-native normalization, which considers dependencies within nodes, edges, and their combination. We define a range of graph-native normal forms and graph object functional dependencies and propose algorithms for transforming graphs accordingly. We evaluate our contributions using a broad range of synthetic and native graph datasets.

Keywords

Knowledge Graphs, Labeled Property Graphs, Knowledge Graph Quality, Normalization, Redundancy

1 Introduction

Labeled property graphs (LPGs) have emerged as a prominent form of knowledge graphs (KGs), seeing extensive use in a variety of contexts, ranging from data integration [22] to fraud detection [23], and are a central abstraction in modern graph processing systems [32]. Especially with their recent popularity as a source of reliable knowledge in machine learning (ML) and artificial intelligence (AI), e.g., to mitigate hallucinations in large language models (LLMs) [21], ensuring the quality of KGs becomes more and more essential [19]. As a first step towards achieving this goal, new standards and schema-level approaches have been proposed, e.g., the Graph Query Language (GQL) [16, 20], PG-SCHEMA [3], PG-KEYS [4], and transformations or alignments between them [2, 31]. However, being able to define schemas is only the first step. We still need to fully understand what a good schema is and how we can improve the schema of a KG. In relational databases, normalization [6, 10, 11] is a well-known technique to transform a database schema into a schema that is commonly perceived as better with respect to a number of desirable properties. Such transformations are traditionally guided by exploiting functional dependencies (FDs) and are described by normal forms, which generally increase data quality by reducing redundancy; the latter not only unnecessarily inflates storage space but also serves as a source of error since all redundant information needs to be kept in sync to ensure consistency.

Originally introduced for the relational data model in the 1970s [10, 11], normalization has recently been adopted to the LPG setting by Skavantzios and Link [35, 36]. In these works, nodes are

considered similar to relations that are being normalized. However, graphs are subject to additional types of redundancies. Consider, for example, the graph depicted in Figure 1a, which captures information about courses and lecturers. In this graph, we see that it might be worthwhile to enforce that *each course is taught using exactly one book*. However, this is difficult to achieve in the current graph because the information is modeled as a property of edges with label *teaches* that connect teachers to courses. This way of modeling the information might have historical reasons but as the graph keeps on growing and evolving, it might lead to errors. In Figure 1a, redundancy is indicated by dotted lines. Hence, to reduce the degree of redundancy and therefore the potential for inconsistencies, the graph should be normalized so that the information about the book is stored together with the course instead of the teaches edges (see Figure 2). In this paper, we build upon the work proposed by Skavantzios and Link [36], which focuses on dependencies within nodes, and extend it to support also dependencies within edges, and dependencies between nodes and edges. Since our approach focuses on both, nodes and edges, we call it *graph-native*. In summary, the contributions of this paper are as follows:

- (i) We propose a new **dependency** type, **Graph Object Functional Dependencies (GO-FDs)**, that expresses dependencies within and between nodes and edges.
- (ii) Building upon the state of the art, we provide graph-native generalizations of **existing normal forms, which we call Graph-Native Normal Forms (GN-NFs)**.
- (iii) We **identify redundancy patterns for LPGs and transformations to remove them. Based thereon, we** introduce graph-native LPG normalization algorithms that transform a given graph into our **GN-NFs**.
- (iv) Our experimental evaluation shows how graph-native normalization guided by GO-FDs **reduces redundancy and affects query performance**.

This paper is structured as follows: Section 2 introduces basic concepts and a running example; Section 3 reviews related work; Section 4 defines GO-FDs for LPGs; Section 5 analyzes redundancies and defines **GN-NFs**; Section 6 presents our normalization algorithms; Section 7 reports evaluation results; and Section 8 concludes with a summary and future work.

2 Preliminaries and Running Example

Since this paper relies on notions from the relational data model, including functional dependencies (FDs), and applies them to labeled property graphs (LPGs), we provide preliminary notions for these based on the literature [1, 3, 8, 16].

Labeled Property Graphs. We assume pairwise disjoint countable sets of labels L , property keys P , constants C , and (graph) object identifiers O . We often refer to object identifiers as (graph) objects:

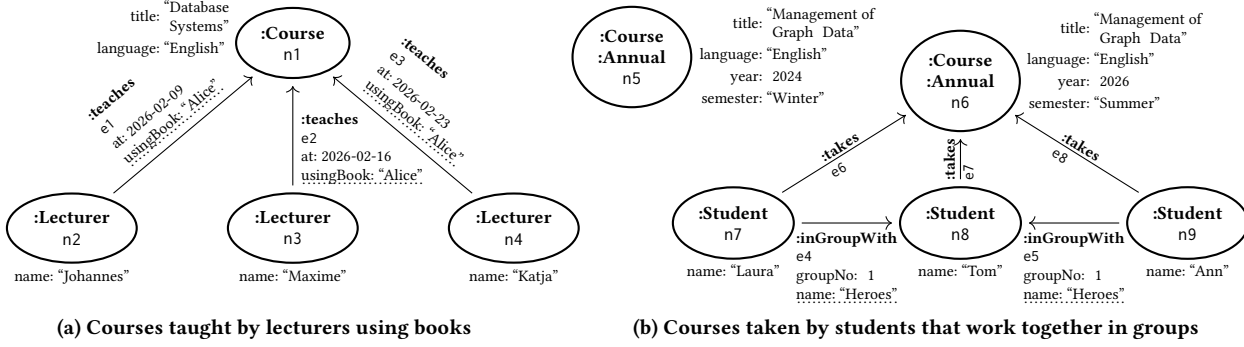


Figure 1: Examples of denormalized LPGs describing courses, lecturers, and students. Redundancies are underlined with dots.

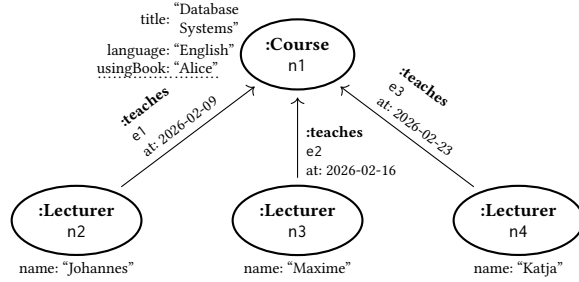


Figure 2: Normalized version of the LPG from Figure 1a

they represent either *nodes* or *edges*. A labeled property graph G is a tuple $(N, E, \text{lab}, \text{src}, \text{tgt}, \text{prop})$ with:

- mutually disjoint finite subsets N, E of $\mathcal{O}$ that represent the nodes and edges respectively;
- a total function $\text{lab} : N \cup E \rightarrow 2^L$ that maps objects to (possibly empty) sets of labels;
- total functions $\text{src} : E \rightarrow N$ and $\text{tgt} : E \rightarrow N$ that map each edge to nodes representing the edge's source and target respectively; and
- a partial function $\text{prop} : (N \cup E) \times \mathcal{P} \rightarrow \mathcal{C}$ that assigns constants to the property keys in objects.

This definition of LPGs is compatible with GQL [20] and common graph database systems, such as Neo4j [26] or Memgraph [25]. The data model of these two systems is a subset of our definition, where, contrary to GQL, edges can only have exactly one label, which is called type.

Relation Schemas, Tuples, and Relations. We assume mutually disjoint countable sets of *relation names* $\mathcal{R}$ and *attributes* $\mathcal{A}$, disjoint with any of the previous sets. When we assume a finite set of attributes $A \subset \mathcal{A}$ with a relation name $R \in \mathcal{R}$, denoted by $\text{attr}(R) = A$, we call R a relation schema with attributes A . A *tuple* μ for a relation schema R is a function that assigns constants from $\mathcal{C}$ to $\text{attr}(R)$. Formally, it is a total function $\mu : \text{attr}(R) \mapsto \mathcal{C}$. An *instance* I of a relation schema R is a set of tuples for R . Sometimes, I is simply called a *relation*. We say that two tuples μ_1, μ_2 agree on attributes A if for every $a \in A$, $\mu_1(a) = \mu_2(a)$.

Functional Dependencies and Keys. Essential to all techniques related to normalization are so-called dependencies. A dependency is a type of constraint that holds on multiple data records – it can cover multiple data attributes [9]. The dependency type relevant for

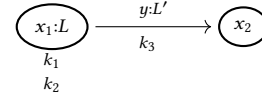


Figure 3: Example of the graphical graph pattern notation

normalization in the relational data model are *functional dependencies* (FDs). FDs express “which attributes are functional dependent on others” [11]. In other words, this means that the values of a certain set of attributes X define the values of another set of attributes Y . Formally, an FD over a set of attributes A is an expression $X \Rightarrow Y$, with $X, Y \subseteq A$. An FD $X \Rightarrow Y$ is *applicable* to a relation schema R if $X \cup Y \subseteq \text{attr}(R)$. We say that an FD $X \Rightarrow Y$ is satisfied by a relation instance I for R , denoted by $I \models X \Rightarrow Y$, if for every two tuples $\mu_1, \mu_2 \in I$, whenever they agree on X , they agree on Y . Given a set of FDs Σ over A , and another dependency $X \Rightarrow Y$ we say that $X \Rightarrow Y$ holds in Σ , denoted by $\Sigma \models X \Rightarrow Y$, if for every instance I of a relation schema R with $\text{attr}(R) = A$ that satisfies all dependencies in Σ , we have $I \models X \Rightarrow Y$. We assume for every relation schema R a (possibly empty) set of FDs $\text{deps}(R)$. A *superkey* of a relation schema R is a set $X \subseteq \text{attr}(R)$ such that $\text{deps}(R) \models X \Rightarrow \text{attr}(R)$. A *key* for R is a (subset-)minimal superkey.

Graph Patterns. In the LPG literature, the notion of graph patterns is often used to express queries. We adopt this notion for our purposes and consider a semantics that corresponds to the *graph pattern calculus* (GPC) [15]. We do not need the full power of GPC, specifically, we do not consider repetitions. Throughout this paper, we use two notations for patterns: (i) a graphical notation and (ii) a concrete syntax.

The graphical notation consists of nodes and edges. Nodes are circles, edges are arrows. Both nodes and edges have a variable (typically x or y), an associated set of labels (typically denoted by L), and a set of property keys (typically $P = \{k_1, k_2, \dots\}$). Edges are directed. Throughout the paper, we explicitly state when edges in our notation represent both directions. Figure 3 illustrates the graphical notation with an example. Here, x_1 matches nodes that have (at least) labels L and property keys k_1 and k_2 . Node x_1 has an outgoing edge y with (at least) labels L' and property key k_3 to another node x_2 without any requirements on its labels. This graphical notation corresponds to the following GPC expression:¹

$$(x_1 : L)_{x_1.k_1=x_1.k_1 \wedge x_1.k_2=x_1.k_2} \xrightarrow{y:L'} y.k_3=y.k_3 (x_2)$$

¹We use the filter condition $x_1.k_1 = x_1.k_1$ to ensure property k_1 is present.

Table 1: The evaluation $\llbracket Q \rrbracket^G$ of pattern Q in graph G

Q	$\llbracket Q \rrbracket^G$
$(x:L:P)$	$\left\{ \begin{array}{l} x \mapsto n; \\ x.k \mapsto k_o \\ \text{for each } k \in P \end{array} \right\} \mid \begin{array}{l} n \in N, \text{lab}(n) \subseteq L, \text{ and} \\ \text{prop}(n, k) = k_o \text{ for each } k \in P \end{array} \right\}$
$() \xleftrightarrow{x:L:P} ()$	$\left\{ \begin{array}{l} x \mapsto e; \\ x.k \mapsto k_o \\ \text{for each } k \in P \end{array} \right\} \mid \begin{array}{l} e \in E, \text{lab}(e) \subseteq L, \text{ and} \\ \text{prop}(e, k) = k_o \\ \text{for each } k \in P \end{array} \right\}$
$(x:L:P) \xleftrightarrow{y:L':P'} ()$	$\left\{ \mu \cup \left\{ \begin{array}{l} y \mapsto e; \\ y.k \mapsto k_o \\ \text{for each } k \in P' \end{array} \right\} \mid \begin{array}{l} \mu \in \llbracket (x:L:P) \rrbracket^G \\ e \in E, \text{src}(e) = \mu(x), \\ \text{lab}(e) \subseteq L', \text{ and} \\ \text{prop}(e, k) = k_o \\ \text{for each } k \in P' \end{array} \right\} \right\}$

Additionally, we introduce a concrete syntax for a subset of GPC, which we call *basic graph patterns* and often simply refer to as *graph patterns*. We assume a countable set of *object identity variables* $\mathcal{V}$, disjoint from $\mathbf{L} \cup \mathbf{P} \cup \mathbf{C}$. Let $x \in \mathcal{V}$. A *property variable* is then an expression of the form $x.k$ with $k \in \mathbf{P}$. A *variable* is either a property- or an object identity variable. Based thereon, we define the syntax for *basic graph patterns* as follows:

$$Q ::= (x:L:P) \mid () \xleftrightarrow{y:L:P} () \mid (x:L:P) \xleftrightarrow{y:L':P'} ()$$

where $\leftrightarrow$ is either $\rightarrow$ or $\leftarrow$, $x, y \in \mathcal{V}$ are object identity variables, $L, L' \subseteq \mathbf{L}$ is a set of labels, and $P, P' \subseteq \mathbf{P}$ is a set of keys. Given a graph pattern Q , we define its *output attributes* $\text{attr}(Q)$ as the set of variables that occur in the pattern. We have:

$$\text{attr}((x:L:P)) := \{x.k \mid k \in P\} \cup \{x\}$$

$$\text{attr}(() \xleftrightarrow{y:L:P} ()) := \{y.k \mid k \in P\} \cup \{y\}$$

$$\text{attr}((x:L:P) \xleftrightarrow{y:L':P'} ()) := \{x.k \mid k \in P\} \cup \{y.k \mid k \in P'\} \cup \{x, y\}$$

For an LPG G and a graph pattern Q , the evaluation of Q in G is the relation $\llbracket Q \rrbracket^G$ for a relation schema R_Q with $\text{attr}(R_Q) = \text{attr}(Q)$, as defined in Table 1.

Graph Transformations. Finally, we adapt the notion of graph transformations from Bonifati [7] for our purposes. A transformation is a pair $(Q_{\text{before}}, Q_{\text{after}})$ where Q_{before} is a graph pattern, and Q_{after} is a graph pattern that contains constructors. There are two kinds of constructors: node identity- and label constructors. Essentially, they are functions mapping sets of values to newly created node identities and labels respectively.² We will use the notation $\langle x \rangle$ and $\underline{x}$ to represent the creation of a new node identity and a new label based on the value x . Consider the first row in Table 4 (with $\text{ID} = \text{ID}_{\text{within-n}}$, disregarding the gray arrows for now). The second column (T) represents Q_{before} and is a single node pattern. It matches nodes x with labels L where the property keys k_1 and k_2 are present. Then, the pattern in the third column (T_{new}) is Q_{after} , which constructs a new node identity based on L and the property key k_1 , and also constructs a new label based on the same values. Furthermore, the property keys (and associated values) k_1 and k_2 are moved from node x to node $\langle Lk_1 \rangle$. The semantics of a single transformation is defined by a *creation phase* in which new graph objects and values are created, followed by a *removal phase* in which graph objects and values are removed. For a set of graph transformations $\mathcal{T}$, first, for each transformation $T \in \mathcal{T}$, the creation phase is executed, and then, for each $T \in \mathcal{T}$, the removal phase is executed.

²These constructors are better known as *Skolem functions* [7].

Table 2: Characteristics of relational normal forms [1, 18]

Relational normal form	Characteristic
1NF	All values in the relation are atomic.
2NF	Non-key attributes of the relation schema are fully dependent on keys.
3NF	Non-key attributes are not transitively dependent on keys.
BCNF	Every FD is trivial or contains a superkey on its left-hand side.

Running Example

To illustrate our concepts and contributions, we introduce a running example (cf. Figure 1). It captures an academic context, where courses are taught by lecturers using certain books. Some courses are taught once, others annually. The example also captures students who are taking courses and may form groups to solve course assignments together.

In this example, we can formulate several dependencies:

φ_{Course} : “Courses are identified by their title and year.”

φ_{Book} : “All lecturers teaching a particular course must use the same textbook.”

φ_{GroupNo} : “The assigned number of a student group determines its name.”

φ_{Annual} : “Annual courses may only be taught in one of the two semesters of a year.”

We can classify these dependencies on the graph objects they use. Dependency φ_{Course} is defined on nodes (Courses) and their property keys (title and year). As this dependency only involves node information, we call it a *within-node* dependency. Dependency φ_{Book} involves nodes (Lecturers) but also property keys from related edges (usingBook). We call φ_{Book} a *between-graph-object* dependency, as it involves both, nodes and edges. The last type of dependency involves only edges and their property keys. Dependency φ_{GroupNo} is such an example, and we call it a *within-edge* dependency. Dependency φ_{Annual} is another example of a within-node dependency.

3 Related Work

Before continuing, we review existing work on (non-)relational normalization and (functional) dependencies for graphs.

3.1 Relational Normalization

The foundations of schema *normalization* are inseparably connected to the *relational model* [10]. The first normal form (1NF) has been proposed together with the relational model by Codd [10] in 1970, followed by the second normal form (2NF) [11], the third normal form (3NF) [11], and the Boyce-Codd Normal Form (BCNF) [12] shortly thereafter. An overview of the characteristics of each relational normal form is shown in Table 2.

The standard process of transforming a relation schema that is not in some desired normal form into a set of relation schemas is called *decomposition*. Decomposing a relation schema brings advantages such as smaller relations, independent information stored separately, and the omission of redundancies [1, 29]. For the relational model, decomposition techniques are primarily concerned with *functional dependencies* (cf. Section 2 and Table 2) to achieve 3NF and BCNF schemas. For FDs, a sound and complete set of reasoning rules, the so-called Armstrong’s Axioms, is known and can be used, for example, to identify redundant dependencies [1, 5, 13].

Normalization is the process of decomposing a given relation schema R in order to reach 2NF, 3NF, and BCNF, effectively splitting R into n new relation schemas $R_1, \dots, R_n$ based on its attributes, meaning $\text{attr}(R) = \bigcup_{i=1}^n \text{attr}(R_i)$. Algorithms that perform decomposition are the *synthesis algorithm* [1], which guarantees 3NF for the decomposed relation schemas, and the *decomposition algorithm* [1], which ensures BCNF. Until today, normalization remains a crucial task when designing (relational) schemas [18], as it omits anomalies (cf. [1]), reduces redundancies and the potential of inconsistencies, and it optimizes the performance of databases [38].

3.2 Non-Relational Normalization

Normalization can be applied beyond the relational model. Thalheim [38] introduced *normalization for the entity-relationship (ER) model*, led by the same objectives as relational normalization: *operation anomalies, inconsistent data, and redundancies should be avoided*. Unlike relational normalization, which considers only single relations (*local normalization*), ER normalization can consider whole diagrams or parts of diagrams (*global normalization*).

ER normalization considers the normalization problem, where the goal is to determine whether a transformation (called “infomorphism” in [38, 39]) ψ exists that can be applied to an ER diagram E such that it fulfills the properties of a particular normal form (cf. [38] for details on the properties). One important property is that these transformations need to be *reversible*, i.e., $\psi_{\text{inv}}(\psi(E)) = E$ holds for ψ and its inverse ψ_{inv} .

Normalization for LPGs was first proposed by Skavantzios and Link [35, 36]. LPGs are normalized using a decomposition procedure. In simplified terms, the principle is to first transform a set of nodes with properties P and labels L into a relation with attributes P . Then, this relation is decomposed, and, finally, the obtained relations are transformed back into an LPG. This approach therefore reduces redundancy by resolving within-node dependencies but does not consider the additional types of dependencies that can occur within edges or graph patterns (i.e., *between-graph-object dependencies*). Hence, in this paper, we propose a generalized approach to close this gap by considering these types of dependencies as well.

3.3 Functional Dependencies for Graphs

In the context of LPG normalization, Skavantzios et al. [35–37] proposed *Graph-tailored Functional Dependencies (gFDs)* to express FDs within nodes, and *Graph-tailored Uniqueness Constraints (gUCs)* to express keys. gFDs and gUCs are defined on a subset of properties of a subset of nodes N that have assigned a set of labels $L \subseteq \mathbf{L}$ and have defined values for property keys $P \subseteq \mathbf{P}$. The semantics of gFDs is similar to the semantics of FDs. For example, the gFD

$$\varphi_{\text{gFD-Annual}} := \{ \text{Course, Annual} \} : \{ \text{title, year, semester} \} : \{ \text{title, year} \} \Rightarrow \{ \text{semester} \}$$

expresses dependency φ_{Annual} (cf. Section 2). Similarly, a gUC

$$\varphi_{\text{gUC-Course}} := \{ \text{Course} \} : \{ \text{title, year} \} : \{ \text{title, year} \}$$

expresses dependency φ_{Course} , i.e., courses are identified by their title and year. Notably, gFDs and gUCs are concerned with nodes. In our terminology, they can only express within-node dependencies.

Besides gFDs and gUC, a variety of other dependencies *applicable to LPGs* has been defined. Including gFDs, Manouvrier and Belhajjame [24] analyzed 13 types of dependencies. However, the

focus of most of these dependency types is on the identification of graph objects, on the specification of integrity constraints involving constant values, or on inference of graph objects. Except for the previously mentioned gFDs, none of the dependency types was designed for the purpose of *normalizing* graphs [24].

Not for LPGs, but for arbitrary n -ary relations Hellings et al. [17] defined *Functional Constraints (FCs)*. They define an axiomatic system to allow reasoning over FCs. In particular, FCs can be used to define FDs over RDF-based graphs.

Since Section 6 defines a *graph-native* approach to normalization based on dependencies involving nodes and edges, we need a dependency type tailored to this task.

4 Graph Object Functional Dependencies

As a first step towards graph-native normalization, we define Graph Object Functional Dependencies (GO-FDs) as a dependency type for LPGs that goes beyond within-node dependencies. A GO-FD φ is an expression of the form:

$$\varphi ::= Q :: X \Rightarrow Y$$

where Q is a graph pattern and X, Y are sets of variables (i.e., object identity- or property variables; cf. Section 2). We refer to the pattern Q of a GO-FD as its *scope*, and to $X \Rightarrow Y$ as its *descriptor* D . We call a GO-FD $Q :: X \Rightarrow Y$ *trivial* if $Y \subseteq X$.

Whether a dependency is a *within- or between-graph-object dependency* simply depends on the number of graph object identity variables involved in the dependency. We make this more precise:

Definition 4.1. We call a GO-FD $\varphi = Q :: X \Rightarrow Y$ a *within-graph-object dependency* when the variables in $X \cup Y$ use exactly one object identity variable x . If x occurs as a node (resp. edge) in the scope Q , we call it a *within-node* (resp. *edge*) dependency. Otherwise, when two object identity variables occur in $X \cup Y$, we call φ a *between-graph-object dependency*.

Recall that graph patterns Q match subgraphs in a given graph G , with the requirement that certain sets of labels L and keys P must be present in the matched graph objects. The result of evaluating a graph pattern in a given graph is a relation where the relation schema consists of attributes that are given by the variables mentioned in the pattern. Then, FD $X \Rightarrow Y$ must hold in this relation.

Formally, a graph G satisfies a GO-FD $Q :: X \Rightarrow Y$ if $\llbracket Q \rrbracket^G \models X \Rightarrow Y$. Furthermore, we call a set of GO-FDs Σ a *GN-Schema*. Therefore, more generally, we say a graph G satisfies a schema Σ , denoted by $G \models \Sigma$ if it satisfies each of the dependencies in Σ .

Example 4.2. By using GO-FDs, we are able to express all textually described dependencies of the running example (cf. Section 2).

$$\varphi_{\text{Course}} : (x : \{ \text{Course} \} : \{ \text{title, year} \}) :: x.\text{title}, x.\text{year} \Rightarrow x$$

$$\varphi_{\text{Book}} : () \xrightarrow{y : \{ \text{Teaches} \} : \{ \text{using} \}} (x : \{ \text{Course} \} : \emptyset) :: x \Rightarrow y.\text{using}$$

$$\varphi_{\text{GroupNo}} : () \xrightarrow{y : \{ \text{inGroupWith} \} : \{ \text{groupNo, name} \}} () :: y.\text{groupNo} \Rightarrow y.\text{name}$$

$$\varphi_{\text{Annual}} : (x : \{ \text{Course, Annual} \} : \{ \text{title, year, sem} \}) :: x.\text{title}, x.\text{year} \Rightarrow x.\text{sem}$$

Dependency φ_{Course} acts as a uniqueness (superkey) constraint and shows how to express gUCs using GO-FDs. Dependency φ_{Book} highlights our generalization of gFDs; we are able to express dependencies *between* graph objects. Similarly, φ_{GroupNo} shows how GO-FDs can express within-edge dependencies. Finally, example φ_{Annual} shows how to express gFDs using within-node GO-FDs.

Table 3: Conditions for $Q \sqsupseteq Q'$

Q	Q'	$Q \sqsupseteq Q'$ if
$(x:L:P)$	$(x':L':P')$	$L' \subseteq L$ and $P' \subseteq P$
$(x:L:P)$	$(x':L':P') \xrightarrow{y:L'':P''} ()$	$L' \subseteq L$ and $P' \subseteq P$
$(x:L_x:P_x) \xrightarrow{y:L_y:P_y} ()$	$(x':L'_x:P'_x) \xrightarrow{y':L'_y:P'_y} ()$	$L'_x \subseteq L_x, P'_x \subseteq P_x;$ $L'_y \subseteq L_y, P'_y \subseteq P_y$

Axiomatization. Functional dependencies in the relational model have an axiomatization called *Armstrong’s axioms*. Analogously, we also have reasoning rules for GO-FDs. First, we state axioms that are intrinsic to LPGs. We call the following three axioms *structurally implied* because they are given by the graph structure of the data model. Let G be a graph.

Object Identity (Node) $G \models (o:\emptyset:k) :: o \Rightarrow o.k$

Object Identity (Edge) $G \models () \xrightarrow{o:\emptyset:k} () :: o \Rightarrow o.k$

Edge Endpoint $G \models (x) \xrightarrow{e} (y) :: e \Rightarrow x$ and $e \Rightarrow y$

Before we continue with the associated reasoning rules, we need to introduce the notion of *pattern inclusion*. Let Q, Q' be basic graph patterns. We say Q is more general than Q' , which we write as $Q \sqsupseteq Q'$ if, for any graph G , $\llbracket Q \rrbracket^G \supseteq \pi_{\text{attr}(Q)}(\llbracket Q' \rrbracket^G)$.³ We characterize basic graph pattern inclusion as follows:

LEMMA 4.3. $Q \sqsupseteq Q'$ iff they are as shown in Table 3.

PROOF SKETCH. One can verify that the relations shown in Table 3 hold. Furthermore, the remaining case is where an edge occurs in Q , but not in Q' . Suppose $Q := (x:L:P) \xrightarrow{y:L':P'} ()$ and $Q' := (x':L'':P'')$. Then, even if $L'' \subseteq L$ and $P'' \subseteq P$, pattern Q is more restrictive since it requires the presence of an edge. $\square$

Furthermore, Table 3 gives us a straightforward algorithm to check whether $Q' \sqsubseteq Q$ by matching the patterns and checking the subsets of property keys and labels.

We now state the reasoning rules for GO-FDs. Let Q, Q' be basic graph patterns.

Reflexivity If $Y \subseteq X$, then $Q :: X \Rightarrow Y$.

Augmentation If $Q :: X \Rightarrow Y$ and $Z \subseteq \text{attr}(Q)$, then $Q :: XZ \Rightarrow YZ$.

Decomposition If $Q :: X \Rightarrow ZY$, then $Q :: X \Rightarrow Y$.

Restriction If $Q :: X \Rightarrow Y$, and $Q' \sqsubseteq Q$, then $Q' :: X \Rightarrow Y$.

Transitivity If $Q :: X \Rightarrow Y$, $Q' :: Y \Rightarrow Z$ and $Q \sqsubseteq Q'$, then $Q' :: X \Rightarrow Z$.

THEOREM 4.4. *The axioms above provide a sound and complete axiomatization of GO-FDs.*

PROOF SKETCH. In case $Q = Q'$ the soundness and completeness of the axiomatization, within the scope Q , follows from Armstrong’s Axioms. However, generally, the completeness of the axiomatization is non-trivial. To show completeness, we rely on the insight that GO-FDs can be encoded into *Functional Constraints (FCs)* over n -ary relations, as defined by Hellings et al. [17]. In their work, they provide a complete axiomatization of FCs, which we can use to show the completeness of our axiomatization. To this end, we must

³We assume the standard relational projection semantics for $\pi_{\text{attr}(Q)}$, i.e., we project on the attributes of Q .

verify that (1) we can encode LPGs in a n -ary relation; (2) we can translate GO-FDs into FCs; and (3) our axioms correspond to theirs.

First, for (1), any LPGs can be encoded into a single relation with arity 4, e.g., as done in *domain graphs* [40]. Then, for (2), we have to show that FCs can express GO-FDs: graph patterns without repetitions can be expressed over this 4-ary relation by ‘patterns’ from Hellings et al. [17], since they are essentially conjunctive queries. Furthermore, their constraint itself is an FD, as are our descriptors. For (3) one can verify that we faithfully adapted Hellings et al.’s [17] axioms in a straightforward way.⁴ $\square$

Obtaining GO-FDs. GO-FDs encode domain knowledge of their scopes and are specified by a curator, typically with implicit knowledge of the graph’s construction and the subgraphs relevant to the application. Skavantzios and Link [36] report that even for within-node dependencies, automated mining gives only heuristics; mining GO-FDs over more complex graph patterns is correspondingly harder, and automatic retrieval of GO-FDs remains an open problem.

5 Redundancies in Labeled Property Graphs

Since one of the main objectives of normalization is to reduce redundancy, we first identify different types of redundancies that might occur in LPGs under GO-FDs and show how these can be resolved in a graph-native way. Based on these insights, we introduce Graph-Native Normal Forms (GN-NFs).

5.1 Redundancy Patterns

Graph data is called *redundant* if it represents the same piece of information *multiple times*.

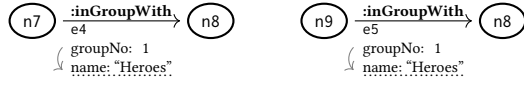
Example 5.1. Consider again the running example in Figure 1b and dependency φ_{GroupNo} , which states that a student group’s number determines the name of the group they are in. The fact that group 1 is named “Heroes” is represented twice, in edges e4 and e5. Figure 4a visually displays the redundancy of the group name in these edges. Preferably, the group name “Heroes” *should* only be represented once in combination with group number 1.

We can characterize these redundancies from a relational perspective. Consider the relation depicted in Figure 4b, which shows the relation represented by the scope of dependency φ_{GroupNo} , including the source and target of the edges. Clearly, if we look at the attributes used in the descriptor of φ_{GroupNo} , i.e., groupNo and name, we can identify the repeated fact.

To simplify our discussion of redundancies, we assume that the dependencies are *strictly* within or between objects, i.e., for a descriptor $X \Rightarrow Y$ at most one object identity variable may appear in *each*, X and Y . We call such dependencies *strict* GO-FDs. They capture natural dependencies, such as the ones used in our examples.

To aid our discussion of redundancy, we discuss the *shape* of a GO-FD: the shape is determined by inspecting the *descriptor* $X \Rightarrow Y$. Specifically, our discussion of redundancy relies on the *types* of variables that are present in X and Y (recall that variables are either identity- or property variables, cf. Section 2).

⁴Interestingly, this argument captures the axiomatization of a generalization of GO-FDs with graph patterns without repetitions. However, such dependencies are out of the scope of this work.



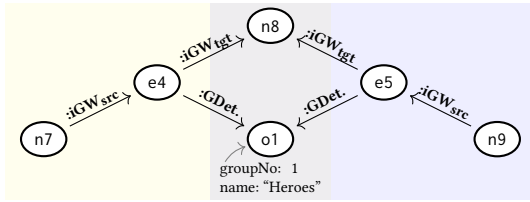
(a) Redundancy within the edges

x	$x.\text{groupNo}$	$x.\text{name}$	$\hat{x}_{\text{src}}$	$\hat{x}_{\text{tgt}}$
e4	1	Heroes	n7	n8
e5	1	Heroes	n9	n8

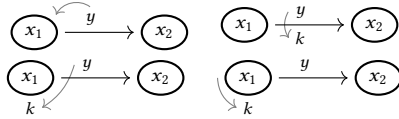
(b) Redundancy from a relational perspective

R_{edge}				R_{groupNo}	
x	$x.\text{groupNo}$	$\hat{x}_{\text{src}}$	$\hat{x}_{\text{tgt}}$	$x.\text{groupNo}$	$x.\text{name}$
e4	1	n7	n8	1	Heroes
e5	1	n9	n8		

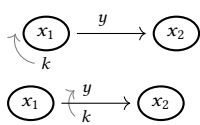
(c) Decomposed relations. The dashed line is a foreign key; the gray line is the descriptor of the original dependency.



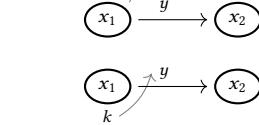
(d) Transformed subgraph of Figure 4a without redundancies

Figure 4: Resolving redundancies in Figure 1 related to φ_{GroupNo} : the original redundancy (a, b), its relational decomposition (c), and the transformed subgraph (d).

(a) Structurally implied GO-FDs



(b) Superkey GO-FDs



(c) Cardinality-limiting GO-FDs

Figure 5: GO-FDs of different shapes for which no redundancies can occur. GO-FD descriptors are shown in gray. For simplicity, only one direction of edges is shown.

First, as shown in Figure 5, we identify three groups of GO-FDs for which no redundancies occur.⁵ Each group contains a number of dependencies of different shapes, using the graphical notation. The source and target of the gray arrow indicate the shape. For simplicity, only one direction is shown for the edges. Still, the same classes also apply to the inverse direction.

THEOREM 5.2. *No redundancies occur under dependencies that follow the shapes shown in Figure 5.*

⁵While only one property key k is depicted in these dependencies, this is only a graphical simplification: generally, these can be sets of property keys.

Table 4: Graphical representation of the graph transformations that resolve redundancy patterns. The descriptor of a GO-FDs is shown as a gray arrow. New nodes are constructed on values x matched in the redundancy pattern (shown using the $\langle x \rangle$ notation), similarly, new labels are constructed (shown using the $\underline{x}$ notation).

ID	Redundancy pattern (T)	Resolved redundancy pattern (T_{new})
<i>Redundancies and transformations associated with within-graph object GO-FDs</i>		
$\psi_{\text{within-n}}$		
$\psi_{\text{within-e}}$		
<i>Redundancies and transformations associated with between-graph object GO-FDs</i>		
$\psi_{\text{between-n-ep}}$		
$\psi_{\text{between-np-ep}}$		
$\psi_{\text{between-ep-np}}$		
$\psi_{\text{between-ep-n}}$		

PROOF. Group (a) shows dependencies that are satisfied by every graph. They can be derived from the structurally implied dependencies in combination with the reasoning rules (cf. Section 4). For such patterns, it is not possible to express the same fact twice: for any given edge, there is exactly one source and target. Similarly, for any graph object, there can be at most one value for a property key. Furthermore, any change in the graph structure will result in a graph that satisfies similar dependencies. Group (b) depicts dependencies that act as graph object (super)keys: either for nodes, or for edges. As for the relational setting, keys are the only kind of dependencies that are desired to be present in your data. Concretely, if k is a superkey for a certain node, this cannot be represented twice due to the nature of superkeys. Finally, group (c) shows dependencies that limit the number of outgoing edges of a node to be at most 1. By definition, such a fact cannot be stated twice. $\square$

All other dependency shapes allow for redundancies to occur. We call dependencies of such shapes *redundancy patterns*. They are listed in the second column of Table 4 (T).

Table 5: Rationale for the redundancy removing transformations

ID	Relation schema of the redundancy pattern	Decomposed relation schema	Transformation rationale
Rationales for within-GO-FD transformations			
$\psi_{\text{within-n}}$	$\{x, x.k_1, x.k_2\}$	$R_1 = \{x, x.k_1\}$	R_1 has one referencing attribute: the node x itself. We conclude that R_1 represents node x . R_2 has no referencing attributes, and therefore, it represents a new node n . Since R_1 has a FK to R_2 , we materialize it as an edge between x and n . Since we no longer need the FK, k_1 needs not be stored in node x .
		$R_2 = \{x.k_1, x.k_2\}$ $x.k_1$ in R_1 is a foreign key (FK).	
$\psi_{\text{within-e}}$	$\{x_1, x_2, y, y.k_1, y.k_2\}$	$R_1 = \{x_1, x_2, y, y.k_1\}$	R_1 has three referencing attributes: edge y , its source and target. We conclude that R_1 represents edge y . R_2 has no referencing attributes, and therefore, it represents a new node n . Since R_1 has a FK to R_2 , we want to materialize it as an edge. Therefore, we reify y , with a reifier node n' . Then, we create the edge between n' and n . Since we no longer need the FK, k_1 needs not be stored in node n' .
		$R_2 = \{y.k_1, y.k_2\}$ $y.k_1$ in R_1 is a FK.	
Rationales for between-GO-FD transformations			
$\psi_{\text{between-n-ep}}$	$\{x_1, x_2, y, y.k_1\}$	$R_1 = \{x_1, x_2, y\}$	R_1 has three referencing attributes: edge y , and its source and target. We conclude that R_1 represents edge y . R_2 has one referencing attribute x_1 , and therefore, we conclude it represents node x_1 . Additionally, R_2 contains a new property key, which represents moving property key k_1 from y to x_1 .
		$R_2 = \{x_1, y.k_1\}$ x_1 in R_1 is a FK.	
$\psi_{\text{between-np-ep}}$	$\{x_1, x_2, y, x.k_1, y.k_2\}$	$R_1 = \{x_1, x_2, y, x.k_1\}$	R_1 has three referencing attributes: edge y , its source and target. We conclude that R_1 represents edge y . R_2 has no referencing attributes, and therefore, it represents a new node n . Since R_1 has a FK using a property from x_1 , we materialize it as an edge between x_1 and n . Since we no longer need the FK, k_1 needs not be stored in node x_1 .
		$R_2 = \{x.k_1, y.k_2\}$ $x.k_1$ is a FK.	
$\psi_{\text{between-ep-np}}$	$\{x_1, x_2, y, x.k_1, y.k_2\}$	$R_1 = \{x_1, x_2, y, y.k_2\}$	R_1 has three referencing attributes: edge y , its source and target. We conclude R_1 represents the edge y . R_2 has no referencing attributes, therefore, it represents a new node n . Since R_1 has a FK using a property key from y , we want to materialize it as an edge. Therefore, we reify y , with a reifier node n' . Then, we create the edge between n' and n . Since we no longer need the FK, k_1 needs not be stored in node n' .
		$R_2 = \{y.k_2, x.k_1\}$ $X.x_1$ is a FK.	
$\psi_{\text{between-ep-n}}$	$\{x_1, x_2, y, y.k_1\}$	$R_1 = \{x_1, x_2, y, y.k_1\}$	R_1 has three referencing attributes: edge y , and its source and target. We conclude that R_1 represents edge y . R_2 has one referencing attribute x_1 , however, since it needs to be referenced from the edge y , an edge will be materialized between the reified node n' and a new node, specially for k_1 .
		$R_2 = \{x_1, y.k_1\}$ $y.k_1$ is a FK.	

5.2 Redundancy Removing Transformations

Since our objective is to eliminate redundant graph data, we propose a general technique that, for each possible redundancy pattern, eliminates the redundancy by specifying a specific graph transformation (cf. Section 2). This is similar to ER normalization [38], where also transformations are employed to avoid modeling that allows for redundancies. The transformations are shown in Table 4, which depicts the redundancy pattern (T), as well as the resolved pattern (T_{new}). Gray arrows denote the dependencies present in the patterns. For example, the transformation $\psi_{\text{between-n-ep}}$ applies to GO-FDs of the form $(x_1:0:0) \xrightarrow{y:L:\{k_1, \dots\}} () :: x_1 \Rightarrow y.k_1$. Concretely, this covers, e.g., dependency φ_{Book} (cf. Section 2).

Generally, the discussed transformations follow a principled approach: elements mentioned in the descriptor $X \Rightarrow Y$ of a GO-FD belong together. More specifically, they are bundled together into a new node n_{new} that is identified by X . This node n_{new} additionally must be reachable from the graph object o of which X was initially a part of. If o is a node, an edge is created to reference n_{new} from o . However, when o is an edge, a different approach must be used.

For descriptors where the left-hand side consists of variables originating from an edge, we employ a *reification pattern*: the edge is replaced by a reifier node n_{reif} , which has an edge to the newly created node n_{new} . These strategies are succinctly summarized as graph transformations in Table 4. We motivate this approach with an example.

Example 5.3. Continuing Example 5.1, we start by observing that relation R resulting from the scope of φ_{GroupNo} , as shown in Figure 4b, has three references to graph objects: edge x , its source and its target. This is characteristic for edges. Nodes, on the other hand, never refer to any other graph object.

We show that the redundant information discussed in Example 5.1 is eliminated by executing the transformation $\psi_{\text{within-e}}$ from Table 4. In the relational setting, we would decompose R into the two relations R_{Edge} and R_{GroupNo} shown in Figure 4c. However, it is not clear from the outset how these two relations correspond to graph objects. Looking at R_{GroupNo} , it refers to no graph objects, which means it is a node in principle. On the other hand, R_{Edge} refers to *three* objects (the edge itself, source, and target), which corresponds to the edge x itself. Since R_{Edge} additionally also has a foreign key to R_{GroupNo} , we would like to capture this relationship by introducing an edge. However, we cannot create an edge where one of the endpoints is an edge itself. We argue that this must be resolved by introducing a *reifier* node ($\langle y \rangle$ in transformation $\psi_{\text{within-e}}$) that can reference nodes from R_{GroupNo} using an edge. Furthermore, since we consider elements of R_{GroupNo} to be nodes, they will need an identifier: in $\psi_{\text{within-e}}$ this identifier is constructed based on the labels and the property keys. Thus, we create an identifier based on the label `inGroupWith` and property key `groupNo`. Additionally, a suitable label l is constructed for this node based on the same label and property. The result after executing this transformation is shown in Figure 4d. Notably, while in relational decomposition R_{GroupNo} the group number is a key, the transformed pattern represents this by adding an explicit key constraint, specifically for nodes with label l where the group number acts as a key for such nodes: $(x:l:\{\text{groupNo}, \text{name}\}) :: \text{groupNo} \Rightarrow x$.

We generalize the arguments from Example 5.3 in Table 5. Essentially, for each entry in Table 4, we look at the relation schema R induced by the redundancy pattern. In particular, we are interested in the number of attributes that reference graph objects. We call such attributes *referencing attributes*. Indeed, when the pattern is a node, there will be exactly one such attribute. Otherwise, if the pattern has an edge, there are three: the source, target, and edge

identity. Given a descriptor $X \Rightarrow Y$ (indicated by the gray arrows in Table 4), we decompose R into two new relation schemas R_1 and R_2 , with $\text{attr}(R_1) = \text{attr}(R) - Y$ and $\text{attr}(R_2) = X \cup Y$. Depending on the number of referencing attributes, we interpret the effect of the transformation, as described in the column “transformation rationale” of Table 5.

The redundancy removing transformations are both lossless and dependency preserving. Table 5 shows the equivalent relational decomposition for which we know these properties hold. Indeed, because the join of the decomposed relations schemas R_1 and R_2 results again in the relation schema of the redundancy pattern, we can say that the transformations are lossless. Furthermore, Table 4 specifies exactly in what way the original dependency is preserved (indicated by the grey arrow in the “Resolved redundancy pattern” column). Specifically, the left-hand side of the descriptor of the original dependency now acts as a key of the newly created nodes. To reconstruct this original dependency from the transformed graph, the (structurally implied) Object Identity reasoning rule needs to be applied.

Expressing the elimination of a redundancy pattern as a graph transformation provides insight into the elimination process itself. When new nodes are created in this process, they may need to be merged together. Recall, that the transformations introduce new identifiers based on existing values on the graph data. Indeed, we want these new identities to overlap in a controlled manner: this expresses the merging of nodes that express the same information.

Example 5.4. Figure 4d shows the subgraph of Figure 4a in which redundancy has been eliminated. The yellow highlight shows the application of the transformation on edge e4, and the blue highlight the transformation of e5. Since the identity of the newly created node o1 (representing the fact that group 1 is called “Heroes”) is the same for edge e4 and e5, the reification nodes both point to o1.

5.3 Graph-Native Normal Forms

While each individual redundancy pattern can be resolved with its associated transformation, our goal is to mitigate all redundancy patterns described above. In this section, we define exactly under which conditions these redundancy patterns are mitigated. We call these the Graph-Native Normal Forms (GN-NFs).

In order to define the GN-NFs, we leverage the ideas of relational normal forms (cf. Table 2, [1, 18]), and ER diagram normalization (cf. Section 3, [38]). Recall the characteristics from the relational normal forms (cf. Table 2). We adopt the definitions of the first two graph-native normal forms, GN-1NF and GN-2NF, directly from their relational counterparts. We formulate them as:

Definition 5.5 (GN-1NF). A graph G is in *GN-1NF* if all property values are atomic, i.e., $\forall c_1, c_2 \in \mathbf{C} : c_1 \notin c_2$.

Indeed, GN-1NF is independent of the dependencies that are present. While – just as in many relational database systems – non-atomic values are supported in graph database systems, e.g., in form of a list datatype, these are typically not considered in classical approaches to normalization. Since our definition of LPGs does not consider non-atomic values, all LPGs satisfy GN-1NF by definition.

Definition 5.6 (GN-2NF). A graph G is in *GN-2NF* with respect to a GN-Schema Σ if every property mentioned in the schema is fully dependent on a key.

Recall the structurally implied axioms (cf. Section 4). Indeed, in any graph, for every property k of graph object o , there is a trivial key: the object o itself. Therefore, we conclude that every GN-Schema vacuously satisfies GN-2NF.

To define GN-3NF and GN-BCNF, we first introduce Q -GN-3NF and Q -GN-BCNF. We call these *scoped* normal forms, in line with our GO-FD terminology, since they are defined relative to a graph pattern Q (the scope).

Definition 5.7 (Q-GN-3NF). A graph G is in *Q-GN-3NF* with respect to a GN-Schema Σ if whenever $\Sigma \models Q :: X \Rightarrow Y$, then either: (1) X is a superkey for $\text{attr}(Q)$, (2) Y is prime, i.e., it is a subset of a key, or (3) $Q :: X \Rightarrow Y$ is trivial.

Similarly, we define Q -GN-BCNF as follows:

Definition 5.8 (Q-GN-BCNF). A graph G is in *Q-GN-BCNF* with respect to a GN-Schema Σ if whenever $\Sigma \models Q :: X \Rightarrow Y$, then X is a superkey for $\text{attr}(Q)$, or $Q :: X \Rightarrow Y$ is trivial.

These are direct adaptations of the well-known relational normal forms to the setting with LPGs and GO-FDs. Furthermore, they generalize the $L:P$ normal forms of [36], which use node patterns for Q and strictly within-node dependencies in Σ .

Ultimately, we want to define a normal form for a GN-Schema without specifying a particular scope Q . In particular, we want a graph with a GN-Schema Σ to be in Q -GN-3NF or Q -GN-BCNF for every Q present in the dependencies of Σ .

Definition 5.9 (GN-3NF and GN-BCNF). A graph G is in *GN-BCNF* (resp. *3NF*) with respect to a GN-Schema Σ if, for every Q present in the dependencies of Σ , it is in Q -GN-BCNF (resp. *3NF*).

Transforming a graph G that satisfies a schema Σ into a graph G' s.t. it satisfies Σ' , which is in GN-BCNF (or 3NF), is the process of *graph-native normalization*. We say a graph (schema) is in Graph-Native Normal Form (GN-NF) if it is either in GN-3NF or GN-BCNF.

6 Graph-Native Normalization

Considering GO-FDs and GN-NFs, we are ready to define a graph-native approach for the normalization of LPGs. This normalization process considers not only within-node but also within-edge and, generally, between-graph-object dependencies. While Section 5 addressed each redundancy pattern individually, this section eliminates them collectively for a given schema, transforming any graph into GN-NF. We first define normalization with respect to a single scope Q (Algorithm 1), and then extend it to full normalization (Algorithm 2).

6.1 Scoped Normalization

We will start with the normalization process to transform a given graph G and GN-schema (i.e., a set of dependencies) such that, for a given scope Q , it satisfies Q -GN-3NF, and may satisfy Q -GN-BCNF.

The process is inspired by the *synthesis* algorithm of relational normalization [1], which always considers all dependencies. An alternative would be the decomposition algorithm from relational normalization [1], which guarantees BCNF but is not always

Algorithm 1: SCOPEDLPGNORMALIZE

The transformation-based approach to graph-native normalization based on the synthesis algorithm

Input: a scope Q ; a GN-schema Σ ; and an LPG G

Phase 1: Determine the set of dependencies Σ_Q relevant to Q

$\Sigma_Q := \emptyset$

for $\varphi : Q' :: X \Rightarrow Y \in \Sigma$ **do**

if $Q \sqsubseteq Q'$ **then**
 $\Sigma_Q := \Sigma_Q \cup \{\varphi\}$

Phase 2: Apply Restriction and compute the minimal cover

$\Sigma_Q^R := \emptyset$

for $\varphi : Q' :: X \Rightarrow Y \in \Sigma_Q$ **do**

$\Sigma_Q^R := \Sigma_Q^R \cup \{Q :: X \Rightarrow Y\}$

$\Sigma_Q^{\min} := \emptyset$

for $\varphi : Q' :: X \Rightarrow Y \in \Sigma_Q^R$ **do**

if φ is minimal and not redundant **then**
 $\Sigma_Q^{\min} := \Sigma_Q^{\min} \cup \{\varphi\}$

Phase 3: Construct transformations and new dependencies

$\mathcal{T} := \emptyset$

$\Sigma_Q^{\text{new}} := \emptyset$

for $\varphi : Q :: X \Rightarrow YZ \in \Sigma_Q^{\min}$ **do**

for $Y' \subseteq YZ$ **do**

for $(T, T_{\text{new}}) \in \mathcal{T}_{\text{red}}$ **do**

if $X \Rightarrow Y' \hat{=} \psi$ **then**

$\mathcal{T} := \mathcal{T} \cup \{(Q, T_{\text{new}})\}$

if $T \neq \psi_{\text{between-n-ep}}$ **then**

$\Sigma_Q^{\text{new}} := \Sigma_Q^{\text{new}} \cup T_{\text{new}} :: X^T \Rightarrow n$

Phase 4: Apply graph transformations

for $\psi \in \mathcal{T}$ **do**

 create(ψ, G)

 % Create new graph objects and set properties as defined by ψ

for $\psi \in \mathcal{T}$ **do**

 remove(ψ, G)

 % Remove properties and graph objects as defined by ψ

Result: Σ_Q^{new} is the schema and G is normalized w.r.t. Q

dependency-preserving and could thus leave redundancies in normalized graphs. The process consists of four phases, shown in Algorithm 1.

Phase 1: Determining Applicable Dependencies. Since we normalize with respect to a particular scope Q , we need to determine which dependencies in Σ apply to that scope so that we can properly compute the minimal cover in the next phase. We know that a graph G satisfies a GN-Schema Σ , i.e., $G \models \Sigma$. Hence, by the definition of GO-FDs, the evaluation of the scope pattern Q in graph G , i.e., $\llbracket Q \rrbracket^G$, satisfies every functional dependency $X \Rightarrow Y$ of a GO-FD $Q :: X \Rightarrow Y$ that is present in Σ . However, these are not the only dependencies $\llbracket Q \rrbracket^G$ satisfies. We also need to consider every GO-FD whose scope Q' is more general than Q , i.e., $Q' \sqsupseteq Q$. Intuitively, this is because a more general scope Q' also “covers” the matched subgraph patterns of the more specific scope Q . Formally, this is an application of the *Restriction* reasoning rule (cf. Section 4). Recall that Theorem 4.3

gives us a straightforward way to determine whether Q' is more general than Q . Thus, the set of applicable dependencies Σ_Q is defined as $\Sigma_Q := \{Q :: X \Rightarrow Y \mid Q' :: X \Rightarrow Y \in \Sigma \text{ and } Q \sqsubseteq Q'\}$.

Phase 2: Minimal Cover. We would like to have a minimal cover $\Sigma_Q^{\min}$ of the set of all dependencies that are relevant for the scope Q . As in relational normalization, this minimal cover is computed to avoid the consideration of redundant or non-minimal GO-FDs for the determination of transformations $\mathcal{T}$ in Phase 3. We say a GO-FD $\varphi : Q :: X \Rightarrow Z$ is (i) redundant if a dependency φ is implied by all other dependencies of a GN-schema Σ , i.e., $\Sigma - \{\varphi\} \models \varphi$, and is (ii) minimal if there is no $Y \subset X$ such that $\Sigma - \{\varphi\} \models Q :: Y \Rightarrow Z$. We use the set of applicable dependencies from Phase 1 (Σ_Q) to compute $\Sigma_Q^{\min}$. After applying *Restriction* (cf. Section 4) to each dependency in Σ_Q , we can reason over them using Armstrong’s Axioms and compute the minimal cover as for FDs [1].

Phase 3: Constructing Transformations. The core of the graph-native normalization approach are the redundancy-removing transformations from Section 5. To construct the set of transformations $\mathcal{T}$ needed for normalization, as the first step, every dependency $\varphi : X \Rightarrow YZ \in \Sigma_Q^{\min}$ with multiple variables on its right-hand side needs to be split up by using the *Decomposition* rule (cf. Section 4) such that only one variable occurs on the right-hand side of φ , i.e., in Y . As next step, based on the property- and object-identity variables (cf. Section 2) used in the descriptor $X \Rightarrow Y$, every GO-FD $\varphi : Q :: X \Rightarrow Y$ is matched with a redundancy pattern T from a transformation $\psi \in \mathcal{T}_{\text{red}}$ shown in Table 4. For example, to match with the transformation $\psi_{\text{between-np-ep}}$, X must consist of a property variable of a node and Y is a property variable of an edge. In Algorithm 1, we use the expression $X \Rightarrow Y \hat{=} \psi$ to denote the matching. Following that, every matched transformation $\psi : (T, T_{\text{new}})$ is adapted to match the pattern Q , and then added to the set of transformations $\mathcal{T}$ that will be applied in Phase 4. As shown in Table 4 and described in Section 5, for transformations that involve the creation of a new node n (all except $\psi_{\text{between-np-ep}}$), a key for n is defined based on the transformed left-hand side X^T of the original dependency φ . For $\psi_{\text{between-np-ep}}$ no new dependency needs to be added explicitly, since it is already structurally implied.

Phase 4: Applying Transformations. Finally, the graph is transformed according to the constructed transformations $\mathcal{T}$. Recall from Section 2 that a set of transformations is applied in two phases: new graph data is created first, then superfluous data is removed. Since each individual transformation is lossless and dependency-preserving, so is the combined application of $\mathcal{T}$.

Example 6.1. We show the application of SCOPEDLPGNORMALIZE (Algorithm 1) and highlight the importance of Phases 1 and 2. For this we consider a GN-Schema Σ , which relies on two scopes that use $P = \{\text{title, program, language}\}$:

$Q_1 : (x : \{\text{Course}\} : P)$

$Q_2 : (x : \{\text{Course, International}\} : P) \xrightarrow{y : \{\text{teaches}\} : \{\text{usingBook}\}} ()$

The considered GN-Schema Σ contains the dependencies:

$\varphi_1 : Q_1 :: x.\text{title} \Rightarrow x.\text{program}$; $\varphi_2 : Q_1 :: x.\text{title} \Rightarrow x.\text{language}$;

$\varphi_3 : Q_2 :: x.\text{program} \Rightarrow x.\text{language}$; $\varphi_4 : Q_2 :: x.\text{language} \Rightarrow y.\text{usingBook}$

and φ_{Course} (cf. Example 4.2)

Algorithm 2: FULLLPGNORMALIZE

Full normalization algorithm for transforming LPGs into GN-NF.

Input: a schema Σ ; and an LPG G

Collect all scopes

$Q := \{Q \mid Q :: X \Rightarrow Y \in \Sigma\}$

Topologically sort the scopes based on $\sqsubseteq$ (see Theorem 4.3)

$Q_{\text{sorted}} := \text{Sort}_{\sqsubseteq}(Q)$

Normalize for each scope

for scope $Q \in Q_{\text{sorted}}$ **do**

 | SCOPEDLPGNORMALIZE(Q, G, Σ) % see Algorithm 1

Result: G is in GN-NF

For this application of Algorithm 1, we decide to perform a scoped normalization regarding scope Q_2 and schema Σ on graph G .

The *first phase* loops over every GO-FD in Σ . Since Q_1 is more general than Q_2 (cf. Table 3), all GO-FDs that use Q_1 are added to Σ_Q . Intuitively, we can see that dependencies with scope Q_1 are applicable to Q_2 : international courses are also courses, and if the necessary property keys are present, the (more general) course dependencies also apply to international courses. Trivially, Q_2 is more general than itself, so also every GO-FD that uses Q_2 is added to Σ_Q . However, dependency φ_{Course} is not applicable for scope Q_2 since it requires the presence of the property keys {title, year}, which is not a subset of P due to year.

Having collected all GO-FDs that hold for scope Q_2 , *Phase 2* then computes the minimal cover of these dependencies. Even though φ_1 , φ_2 , φ_3 , and φ_4 are present in Σ_Q , the minimal cover $\Sigma_Q^{\min}$ will not include φ_2 , as it can be implied by the other three dependencies.

In *Phase 3* the graph transformations are determined: $\psi_{\text{within-n}}$ for both φ_1 and φ_3 , and $\psi_{\text{between-np-ep}}$ for φ_4 . Indeed, if we had kept φ_2 in *Phase 3*, the transformation resulting from that dependency would lead to the creation of a superfluous new node. Thus, *Phase 2* prevents the creation of such redundant nodes.

Finally, in *Phase 4*, the transformations are applied to G .

6.2 Full Normalization

Algorithm 1 normalizes with respect to one scope, but GN-NF (Definition 5.9) requires normalization for every scope Q . Full graph-native normalization (Algorithm 2, FULLLPGNORMALIZE) achieves this by repeatedly applying SCOPEDLPGNORMALIZE (Algorithm 1) to each scope. The order of application is crucial: scopes must be processed from least to most general to avoid side-effects. We therefore topologically sort the scopes in Σ using the “more general than” partial order ($\sqsubseteq$, cf. Theorem 4.3). The following example illustrates why more specific scopes must be considered first:

Example 6.2. Consider the situation from Example 6.1, where we normalized within the scope Q_2 only. We now normalize for all scopes and show the importance of the order of scopes for full normalization. Assume we consider the scope Q_1 first. This would transform every course according to the transformation rules. For example, by φ_1 , the properties title and program would be moved from the course nodes to a new node due to transformation rule $\psi_{\text{within-n}}$. Specifically, this means that the *international* courses would also be affected by this transformation. However, as a side effect, this movement of properties makes the GO-FDs φ_3 and φ_4 no longer applicable. In particular, in this example, if we want to

normalize with scope Q_2 after Q_1 , transformations resulting from φ_3 and φ_4 would have no effect.

This is why the topological ordering is crucial. Executing normalization with the more specific scope Q_2 first avoids this situation. Then, in the beginning, only international courses will be modified, considering the dependencies relating to regular courses as well (due to the application of Restriction in Phase 1 of SCOPEDLPGNORMALIZE). Afterward, normalization with the more general scope Q_1 only affects course nodes, as international courses have already been normalized and no longer match the original dependencies.

The result is a graph that is normalized with respect to Σ and every scope Q . Hence G is in GN-NF.

7 Evaluation

Technical Setup. We implemented our approach in Python utilizing an ANTLR grammar [30] to parse GO-FDs defined on graph patterns; it supports graphs stored in Neo4j and Memgraph. Our implementation and evaluation setup incl. Docker, artifacts, etc. are available on GitHub⁶. All experiments were run on a server with a 24-core 2.9 GHz AMD EPYC 9254 CPU and 768 GB DDR5 RAM. Neo4j was configured to use a page cache of 700 GB and a heap of 31 GB, as recommended by Neo4j.

Metrics. An overview of our metrics and their definitions are provided in Table 6. We distinguish between metrics measuring the structure and size of an LPG and metrics that measure query performance. Regarding graph structure and size, on a per-graph basis, we measure the number of nodes ($m_{\#Node}$) and edges ($m_{\#Edge}$), as well as the average number of properties per node ($m_{\text{AvgPropNode}}$) and edge ($m_{\text{AvgPropEdge}}$). For a set of GO-FDs Σ , i.e., a GN-Schema, we measure the number of all GO-FDs ($m_{\#Dep.sTotal}$) as well as within-node GO-FDs ($m_{\#Dep.sWithinNode}$), within-edge GO-FDs ($m_{\#Dep.sWithinEdge}$), and between-GO-FDs ($m_{\#Dep.sBetween}$).

In line with [36], we also calculate the ‘potentials for redundancy’ for each GO-FD. For a given GO-FD $Q :: X \Rightarrow Y$, and a graph G , the list of redundancy potentials M is calculated by a group-by-count query $M := \mathcal{G}_{\text{count}(X \cup Y)}(\llbracket Q \rrbracket^G)$, effectively counting how often the same combinations of values for property keys in X and Y occur. The higher the redundancy potentials (i.e., the counts in M) are, the more redundancy is present per value combination. If there are no redundant combinations of values, every value in M is 1. Based on M we calculate $m_{\text{MaxRed.Pot.}}$ as the maximum redundancy potential per GO-FD, and $m_{\text{AvgRed.Pot.}}$ as the arithmetic mean of the redundancy potentials M per GO-FD. Due to being based on counts, these two metrics do not have an upper limit.

Another structural metric we define per GO-FD is minimality $m_{\text{Minim.}}$ [14]. It complements the redundancy-potential-based metrics, as it calculates the ratio between the number of distinct and the total number of all combinations of values for property keys in X and Y . Thus, its result values lie between 0 (completely redundant, there is only a single combination of values for property keys in X and Y) and 1 (no redundancy at all, “minimal” in the sense that values for property keys in X and Y only appear in unique combinations). An example calculation of the per-dependency metrics is shown below in Example 7.1.

⁶<https://github.com/dmki-tuwien/lpg-normalization>

Table 6: Definitions of evaluation metrics

	Type	Ref.	Name	Definition
Graph size and structure	Per graph G	$m_{\#Node}$	NodeCount	$ N $
		$m_{\#Edge}$	EdgeCount	$ E $
		$m_{AvgPropNode}$	AvgNodePropCount	$\text{avg}\left(\left \left\{\{p \in P : (n, p) \in \text{dom}(\text{prop})\} : n \in N\right\}\right \right)$
		$m_{AvgPropEdge}$	AvgEdgePropCount	$\text{avg}\left(\left \left\{\{p \in P : (e, p) \in \text{dom}(\text{prop})\} : e \in E\right\}\right \right)$
	Per GN-Schema Σ	$m_{\#Dep.sTotal}$	AllDep.sCount	$ \varphi \in \Sigma $
		$m_{\#Dep.sWithinNode}$	WithinNodeDep.sCount	$ \varphi \in \Sigma \text{ where } \varphi \text{ is a within-node dependency} $
		$m_{\#Dep.sWithinEdge}$	WithinEdgeDep.sCount	$ \varphi \in \Sigma \text{ where } \varphi \text{ is a within-edge dependency} $
		$m_{\#Dep.sBetween}$	BetweenG.O.Dep.sCount	$ \varphi \in \Sigma \text{ where } \varphi \text{ is a between-graph-object dependency} $
	Per GO-FD φ	$m_{MaxRed.Pot.}$	MaxRedundancyCount (cf. metric “#red” in [36])	$\max(\mathcal{G}_{count(D)}(\llbracket Q \rrbracket^G))$
		$m_{AvgRed.Pot.}$	AvgRedundancyCount	$\text{avg}(\mathcal{G}_{count(D)}(\llbracket Q \rrbracket^G))$
		m_{Minim}	Minimality (based on [14])	$\begin{cases} 1, & \text{if } \llbracket Q \rrbracket^G = 1 \\ \frac{ \mathcal{G}_{count(D)}(\llbracket Q \rrbracket^G) - 1}{ \llbracket Q \rrbracket^G - 1}, & \text{else} \end{cases}$
Query performance		$m_{Q.Dur.}$	Query duration (execution time)	As returned by the graph system
		$m_{Q.DBHit}$	Database hits	As returned by the graph system

Table 7: Graphs used in the evaluation

Graph	Source	$m_{\#Node}$	$m_{\#Edge}$	$m_{AvgPropNode}$	$m_{AvgPropEdge}$
London Public Transport	[33]	651	1952	≈ 5	3
Northwind	[27]	1035	3139	≈ 13.30	≈ 3.43
No Socks	[36]	3	2	≈ 3.33	0
Offshore	[28]	2016 523	3339 267	≈ 9.86	≈ 3.25
Train Services	[34]	194 179	1826 601	≈ 6.98	≈ 3.79
University	Figure 1a	4	3	1.25	2

Example 7.1. For the calculation of the per-dependency metrics, $m_{MaxRed.Pot.}$, $m_{AvgRed.Pot.}$, and m_{Minim} we consider an graph G , shown in Figure 6, and a GO-FD

$$\varphi : (x : \{X\} : \{k_x\}) \xrightarrow{y:\{Y\}:\{k_y\}} () :: x.k_x \Rightarrow y.k_y$$

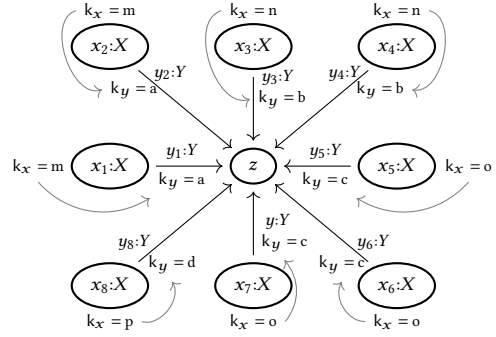
that holds eight times in G (denoted by gray arrows). With respect to φ , the following redundancy potentials are present in G : $M_\varphi = [2, 2, 3, 1]$. This is the result of $(x.k_x, y.k_y) = (m, a)$ occurring twice, $(x.k_x, y.k_y) = (n, b)$ occurring twice, $(x.k_x, y.k_y) = (o, c)$ occurring three times, and $(x.k_x, y.k_y) = (p, d)$ occurring once in G .

The metrics are then calculated as follows:

- $m_{MaxRed.Pot} = \max(M_\varphi) = 3$
- $m_{AvgRed.Pot} = \text{avg}(M_\varphi) = 2$
- $m_{Minim} = \frac{|M_\varphi| - 1}{\text{sum}(M_\varphi) - 1} \approx 0.43$

While the results for $m_{MaxRed.Pot}$ and $m_{AvgRed.Pot}$ might look acceptable, they only provide a limited view on redundancy, as they are statistics calculated on the list of redundancy potentials. As both of these two metrics do not have an upper limit, the presence of redundancies is the only statement that can be made based on them. Contrary, the minimality value always lies between 0 and 1. In this example, it expresses that $\approx 57\%$ of the matches of the dependency φ are redundant value combinations.

For measuring query performance, we rely on the query execution time in milliseconds ($m_{Q.Dur.}$) and on the number of database hits ($m_{Q.DBHit}$), which is a graph system internal metric to express query effort. The absolute values for $m_{Q.Dur.}$ depend on the computer

**Figure 6: Example graph for the calculation of per-dependency metrics**

the LPG system is running on, $m_{Q.DBHit}$ is machine-independent. For both metrics lower values denote better performance.

Test scenarios. We evaluate our approach using six graphs (see Table 7 for details). To compare with related work, we use the same graphs as in [36] and add additional ones to evaluate additional types of dependencies. As shown in Table 8, they are used in 12 scenarios with different GN-Schemas that cover all transformations discussed in this paper (see Table 4). In lack of appropriate methods for mining constraints in graphs, we follow the approach described by [36] and manually define dependencies for each graph⁷ and compute the minimal cover for each GN-Schema. The normalization of each test scenario was performed three times on fresh instances of Neo4j Enterprise, version 2025.12.1, and Memgraph 3.7.2. Query performance experiments were run ten times and the average of all runs is reported.

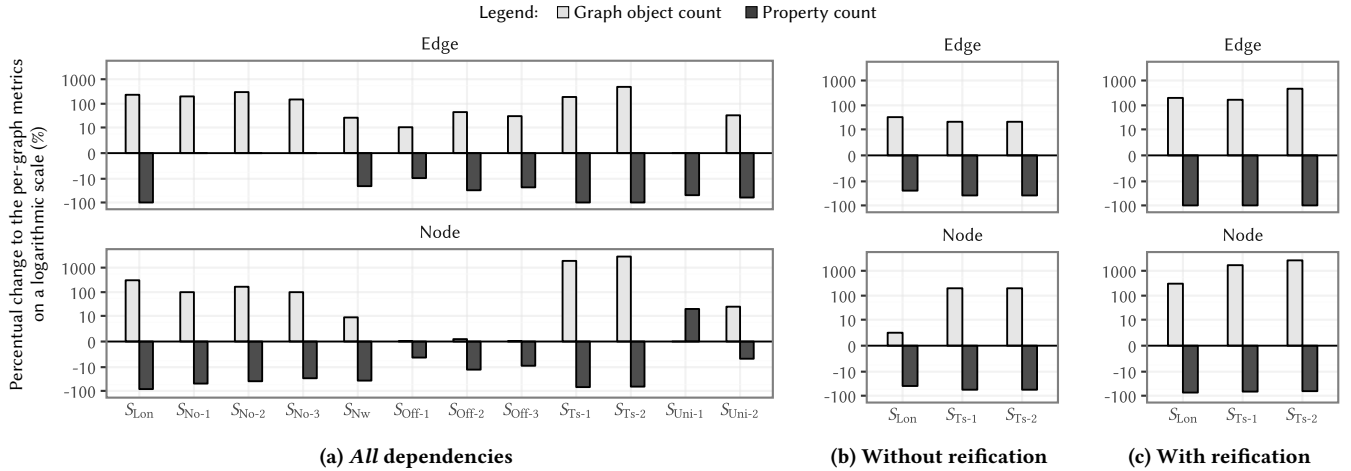
Redundancy Reduction

Table 9 shows the results of our per-GO-FD metrics for each scenario before and after normalization. It shows that originally there

⁷The set of dependencies are available online: <https://github.com/dmki-tuwien/lpg-normalization/blob/master/setup.yaml>

Table 8: Overview of the evaluation scenarios. Durations are in s, are rounded to two decimal places.

Scenario ID	Graph	Provided GO-FDs			Minimal cover of GO-FDs			Applicable transformations	Note	Normalization duration	
		$m_{\text{Dep.sW-Node}}$	$m_{\text{Dep.sW-Edge}}$	$m_{\text{Dep.sBetween}}$	$m_{\text{Dep.sW-Node}}$	$m_{\text{Dep.sW-Edge}}$	$m_{\text{Dep.sBetween}}$			Memgraph	Neo4j
S_{Lon}	London Public Transport	3	2	0	2	1		0 $\psi_{\text{within-n}}$, $\psi_{\text{within-e}}$		0.11	1.33
S_{Nw}	Northwind	2	0	0	2	0		0 $\psi_{\text{within-n}}$	as in [36]	0.04	1.14
$S_{\text{No-1}}$	No Socks	4	0	0	4	0		0 $\psi_{\text{within-n}}$	as in [36]	0.03	1.17
$S_{\text{No-2}}$	No Socks	4	0	0	4	0		0 $\psi_{\text{within-n}}$	as in [36]	0.03	1.41
$S_{\text{No-3}}$	No Socks	5	0	0	5	0		0 $\psi_{\text{within-n}}$	as in [36]	0.02	1.45
$S_{\text{Off-1}}$	Offshore	3	0	0	2	0		0 $\psi_{\text{within-n}}$	as in [36]	9.18	24.77
$S_{\text{Off-2}}$	Offshore	5	0	0	5	0		0 $\psi_{\text{within-n}}$	as in [36]	33.92	116.28
$S_{\text{Off-3}}$	Offshore	10	0	0	5	0		0 $\psi_{\text{within-n}}$	as in [36]	25.15	63.26
$S_{\text{Ts-1}}$	Train Services	3	1	2	2	1		2 $\psi_{\text{w-n}}$, $\psi_{\text{w-e}}$, $\psi_{\text{b-np-ep}}$, $\psi_{\text{b-ep-n}}$		60.89	130.27
$S_{\text{Ts-2}}$	Train Services	3	1	3	2	1		3 $\psi_{\text{w-n}}$, $\psi_{\text{w-e}}$, $\psi_{\text{b-np-ep}}$, $\psi_{\text{b-ep-n}}$, $\psi_{\text{b-ep-np}}$		104.16	408.72
$S_{\text{Uni-1}}$	University	0	0	1	0	0		1 $\psi_{\text{between-n-ep}}$	cf. Figure 1a	0.01	0.19
$S_{\text{Uni-2}}$	University	0	0	1	0	0		1 $\psi_{\text{between-np-ep}}$	adapted from Figure 1a	0.01	0.61

**Figure 7: Percentual changes (log scale) to our structural per-graph metrics following graph-native LPG normalization grouped by graph object, considering all dependencies, only those leading to transformations with reification, and only those without reification.****Table 9: The results of the per-GO-FD metrics before and after graph-native LPG normalization**

Scenario	$\text{avg}(m_{\text{MaxRed.Pot.}})$		$\text{avg}(m_{\text{AvgRed.Pot.}})$		$\text{avg}(m_{\text{Minim}})$	
	before	after	before	after	before	after
S_{Lon}	112.60	1	≈ 36.42	1	≈ 0.41	1
S_{Nw}	16.00	1	≈ 5.16	1	≈ 0.55	1
$S_{\text{No-1}}$	1.25	1	1.25	1	0.75	1
$S_{\text{No-2}}$	1.25	1	1.25	1	0.75	1
$S_{\text{No-3}}$	1.20	1	1.20	1	0.80	1
$S_{\text{Off-1}}$	$\approx 264\,881.33$	1	≈ 613.07	1	≈ 0.00	1
$S_{\text{Off-2}}$	175\,898.00	1	≈ 513.25	1	≈ 0.00	1
$S_{\text{Off-3}}$	239\,572.20	1	$\approx 21\,645.63$	1	≈ 0.00	1
$S_{\text{Ts-1}}$	≈ 6790.33	1	≈ 563.93	1	≈ 0.68	1
$S_{\text{Ts-2}}$	11\,631.00	1	≈ 964.82	1	≈ 0.59	1
$S_{\text{Uni-1}}$	3.00	1	3.00	1	0.00	1
$S_{\text{Uni-2}}$	3.00	1	3.00	1	0.00	1

are redundancies in all scenarios. After performing graph-native normalization, i.e., performing the redundancy removing transformations (cf. Section 5), all redundancies are eliminated. Thus, the per-GO-FD metrics $m_{\text{MaxRed.Pot.}}$, $m_{\text{AvgRed.Pot.}}$, and m_{Minim} all return 1, as for each dependency $\varphi : Q :: X \Rightarrow Y$ there is only a single combination of values for the property keys in X and Y .

Effects of Graph-Native Normalization on LPGs Structure

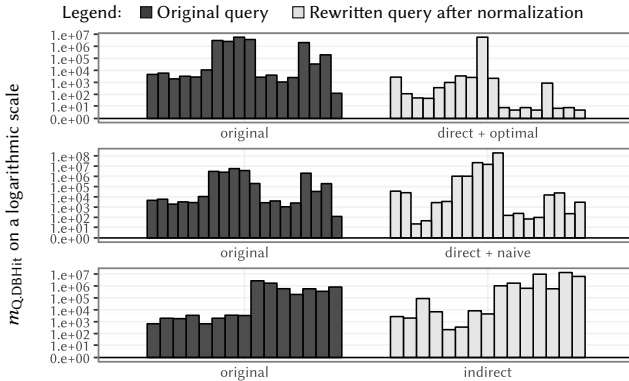
Graph-native LPG normalization affects the structure and size of an LPG, with the precise impact depending on the transformations applied. Figure 7 shows the relative percentage change of our per-graph metrics on a logarithmic scale, broken down by dependency type: Figure 7a for all dependencies, Figure 7c for those leading to reification, and Figure 7b for those that do not. This reveals how different redundancy patterns (cf. Table 4) influence normalization.

Transformations that create new nodes without reification (e.g., $\psi_{\text{within-n}}$ and $\psi_{\text{between-np-ep}}$) introduce new nodes for the properties of the descriptor of a dependency and connect them to the existing subgraph via an edge. As Figure 7b shows, this increases the number of nodes and edges while decreasing the number of properties – an effect of storing each piece of information only once.

Transformations involving reification (e.g., $\psi_{\text{within-e}}$, $\psi_{\text{between-ep-np}}$, and $\psi_{\text{between-np-ep}}$) have the most significant effect on per-graph metrics. As Figure 7c shows, they also reduce

Table 10: Comparison of per-graph metrics after normalizing using our approach and the approach from Skavantzios and Link [36]

Scenario	Approach	$m_{\#Node}$	$m_{\#Edge}$	$m_{AvgPropNode}$	$m_{AvgPropEdge}$
S_{No-1}	Graph-Native	6	6	≈ 1.67	0
	[36]	8	8	2	0
S_{No-2}	Graph-Native	8	8	2	0
	[36]	1124	3969	≈ 8.30	≈ 2.71
S_{No-3}	Graph-Native	6	5	≈ 2.33	0
	[36]	1124	3969	≈ 8.30	≈ 2.71
S_{Nw}	Graph-Native	2 017 381	3 681 864	≈ 9.52	≈ 2.95
	[36]	2 017 381	3 681 864	≈ 9.52	≈ 2.95
S_{Off-1}	Graph-Native	2 021 602	4 854 231	≈ 8.60	≈ 2.24
	[36]	2 021 602	4 854 231	≈ 8.60	≈ 2.24
S_{Off-2}	Graph-Native	2 017 388	4 367 058	≈ 9.01	≈ 2.49
	[36]	2 017 388	4 367 058	≈ 9.01	≈ 2.49

**Figure 8: Comparison of query performance (m_{QDBHit}) between original and rewritten queries**

the average number of properties but, due to reification, create substantially more new nodes and edges.

Overall, Figure 7a shows that graph-native normalization almost always increases the number of graph objects and decreases the average number of properties per object. The exception is $\psi_{between-n-ep}$, which “moves” a property from an edge to a node, as in S_{Uni-1} . There, $m_{\#Node}$ and $m_{\#Edge}$ remain constant while $m_{AvgPropEdge}$ decreases and $m_{AvgPropNode}$ increases. This reflects the movement of the property values from edges to unique nodes.

With respect to runtime, we observed three influencing factors: the number of graph objects before normalization, the number of dependencies in the minimal cover of the GN-Schema to consider, and the type of graph transformations applied during normalization. The latter correlates with the number of graph objects involved: transformations using reification are the slowest. For example, in S_{Lon} on Neo4J, the transformation of type $\psi_{within-n}$ takes ≈ 0.58 s on average, while the transformation of type $\psi_{within-e}$ takes ≈ 0.97 s.⁸

Effects of Graph-Native Normalization on Query Performance

To get insight in the precise influence, we consider the scenarios (graph and dependency combinations, cf. Table 8) and we define queries for each of these scenarios. These queries are formulated using the structure of the graph *before* normalization, we will refer to this as the *original graph*. To properly guide the discussion on

the effect of normalization on querying, we designed two types of queries, which we call *direct* and *indirect*. Direct queries are formulated specifically for a given GO-FD and ask for pieces of information that are redundant in the original graph, but are unique in the normalized graph. Formally, this corresponds to queries which only ask for property keys $k \in X \cup Y$ of a single GO-FD $\varphi : P :: X \Rightarrow Y$. Indirect queries go beyond querying redundant pieces of information, and thus require some more complex navigation.

For all queries defined on the original graph, we need to have a counterpart for the normalized graph. For direct queries, we consider two kinds of counterparts. First, the *optimal* rewriting: these queries only need to consider the newly created graph objects (these represent the normalized non-redundant pieces of information). On the other hand, we have the *naive* rewriting: considering the fully transformed scope which requires additional navigation. Given the new structure, this navigation is not strictly necessary.

The hypothesis is that direct queries improve in performance under optimal rewriting. For all other queries, it depends on the precise graph and query structure.

Table 11 shows this impact for 17 lookup and update queries⁹ executed on four different scenarios, listing execution time and database hits per query before and after normalization. The best performance per query and metric is underlined.

The results confirm our hypothesis. For direct queries with optimal rewriting, performance consistently improves over the original graph, both in terms of execution time and database hits. Two mechanisms drive this improvement: first, the elimination of duplicate information directly reduces the number of database hits required to retrieve a given fact; second, the keys introduced by normalization on the newly created nodes are backed by automatic indices, which further accelerates lookups. The contrast between S_{Lon-1a} and S_{Lon-1b} illustrates the importance of optimal rewriting: both queries retrieve the same result, but S_{Lon-1a} retains the unnecessarily large pattern of the fully transformed scope and in fact performs *worse* than on the original graph, while S_{Lon-1b} uses the optimal rewriting and benefits from the performance gains described above. For indirect queries and direct queries under naive rewriting, the picture is mixed: some queries benefit from the reduced redundancy, while others – such as S_{Lon-2} – suffer from the additional navigation introduced by new graph objects and reification nodes. Regarding the query type, we observed no systematic difference between retrieval and update queries.

We conclude that there is a mixed picture regarding the effects of normalization on query performance (across all types of transformations). However, for direct queries with optimal rewriting, performance does improve. We argue that this is a valuable insight: when a dependency is defined, it typically means that the affected information is of importance. Thus, one might often explicitly query for that information. In practice, direct queries with optimal rewriting might therefore not only be a theoretical idea, but a class of queries that naturally arises whenever the dependencies reflect the meaning of the data.

⁸Detailed results can be found at <https://github.com/dmki-tuwien/lpg-normalization>

⁹The queries can be found in the appendix and on GitHub [Insert link](#)

Table 11: Effects of graph-native normalization on query performance. Better results per graph system are underlined. Execution times are the average of 10 runs and are rounded to two decimal places.

Scenario	Query type	Query no.	Affected by transformation type	Direct	Optimal rewriting	Execution time (in ms)				Database hits			
						before		after		before		after	
						Memgraph	Neo4j	Memgraph	Neo4j	Memgraph	Neo4j	Memgraph	Neo4j
S _{Lon}	retrieval	1a	$\psi_{\text{within-e}}$	✓	✓	2.51	3.94	<u>0.47</u>	<u>0.37</u>	4589	5857	<u>2736</u>	<u>113</u>
		1b	$\psi_{\text{within-e}}$	✓	×	<u>2.51</u>	<u>3.94</u>	10.81	16.09	<u>4589</u>	<u>5857</u>	35 898	25 517
		2	$\psi_{\text{within-e}}$	×	×	0.33	0.88	0.68	<u>0.57</u>	677	1971	2709	2030
		3a	$\psi_{\text{within-n}}$	✓	✓	1.19	1.29	<u>0.09</u>	<u>0.37</u>	1964	3241	50	<u>46</u>
		3b	$\psi_{\text{within-n}}$	✓	×	1.19	1.29	<u>0.04</u>	<u>0.24</u>	1964	3241	<u>22</u>	<u>46</u>
	update	4	$\psi_{\text{within-e}}, \psi_{\text{within-n}}$	×	×	<u>0.71</u>	<u>1.71</u>	13.92	1.90	<u>1824</u>	<u>3459</u>	88 164	6857
		5	$\psi_{\text{within-e}}$	×	×	0.31	0.76	<u>0.12</u>	<u>0.44</u>	670	1971	<u>217</u>	<u>355</u>
		1a	$\psi_{\text{within-e}}$	✓	×	1.09	1.98	<u>0.12</u>	<u>0.38</u>	2706	4001	<u>152</u>	<u>243</u>
		1b	$\psi_{\text{within-e}}$	✓	×	1.09	1.98	<u>0.13</u>	<u>0.35</u>	2706	4001	<u>156</u>	<u>149</u>
		1c	$\psi_{\text{within-e}}$	✓	✓	1.09	1.98	<u>0.07</u>	<u>0.29</u>	2706	4001	<u>7</u>	<u>4</u>
S _{Nw}	retrieval	1	$\psi_{\text{within-n}}$	×	×	<u>1.52</u>	<u>7.51</u>	1.80	12.33	<u>3531</u>	<u>3321</u>	8267	4507
		2a	$\psi_{\text{within-n}}$	✓	×	1.44	4.56	<u>0.76</u>	<u>1.73</u>	2790	10 791	<u>2765</u>	<u>3559</u>
		2b	$\psi_{\text{within-n}}$	✓	✓	1.44	4.56	<u>0.24</u>	<u>0.97</u>	2790	10 791	<u>362</u>	<u>980</u>
	update	1a	$\psi_{\text{within-n}}$	✓	×	0.28	1.28	<u>0.06</u>	<u>0.51</u>	1079	2529	65	<u>98</u>
		1b	$\psi_{\text{within-n}}$	✓	✓	0.28	1.28	<u>0.02</u>	<u>0.40</u>	1079	2529	<u>7</u>	<u>4</u>
		1c	$\psi_{\text{within-n}}$	✓	×	0.28	1.28	<u>0.07</u>	<u>0.53</u>	1079	2529	<u>69</u>	<u>62</u>
S _{Off-1}	retrieval	1a	$\psi_{\text{within-n}}$	✓	×	923.41	406.97	<u>170.21</u>	<u>89.19</u>	3 027 613	2 524 955	<u>1 030 373</u>	<u>1 031 379</u>
		1b	$\psi_{\text{within-n}}$	✓	✓	923.41	406.97	<u>1.11</u>	<u>2.12</u>	3 027 613	2 524 955	<u>3438</u>	<u>2575</u>
		2	$\psi_{\text{within-n}}$	×	×	797.36	<u>254.31</u>	<u>283.41</u>	282.81	2 702 775	<u>1 715 964</u>	<u>1 030 567</u>	1 716 575
	update	1a	$\psi_{\text{within-n}}$	✓	✓	336.04	18.27	<u>0.71</u>	<u>0.28</u>	2 026 155	33 692	866	6
		1b	$\psi_{\text{within-n}}$	✓	×	336.04	18.27	<u>1.97</u>	<u>2.93</u>	2 026 155	33 692	<u>15 310</u>	<u>14 447</u>
		1c	$\psi_{\text{within-n}}$	✓	×	336.04	18.27	<u>4.07</u>	<u>8.14</u>	2 026 155	33 692	<u>15 306</u>	<u>24 071</u>
S _{Ts-2}	retrieval	1a	$\psi_{\text{between-ep-np}}, \psi_{\text{between-np-ep}}$	✓	✓	1075.67	377.66	<u>301.39</u>	<u>1.66</u>	5 675 074	3 654 287	<u>5 673 812</u>	<u>2169</u>
		1b	$\psi_{\text{between-ep-np}}, \psi_{\text{between-np-ep}}$	✓	×	<u>1075.67</u>	377.66	2122.51	<u>1.47</u>	<u>5 675 074</u>	<u>3 654 287</u>	21 532 367	14 623 422
		2	$\psi_{\text{between-ep-np}}, \psi_{\text{between-np-ep}}, \psi_{\text{within-n}}$	×	×	<u>81.39</u>	62.83	8231.41	<u>0.04</u>	<u>200 599</u>	<u>589 524</u>	191 562 509	637 897
		3	$\psi_{\text{within-n}}$	×	×	<u>78.94</u>	57.43	1042.67	<u>54.53</u>	<u>194 463</u>	581 320	9 712 778	<u>578 976</u>
		4	$\psi_{\text{within-n}}$ (twice)	×	×	<u>139.21</u>	<u>116.81</u>	1226.04	514.43	<u>356 326</u>	<u>817 234</u>	13 381 704	6 310 756
	update	1a	$\psi_{\text{between-ep-np}}, \psi_{\text{between-np-ep}}$	✓	×	30.60	0.90	<u>0.09</u>	<u>0.00</u>	194 278	<u>123</u>	<u>230</u>	2960
		1b	$\psi_{\text{between-ep-np}}, \psi_{\text{between-np-ep}}$	✓	×	30.60	0.90	<u>0.11</u>	<u>0.00</u>	194 278	<u>123</u>	<u>226</u>	2989
		1c	$\psi_{\text{between-ep-np}}, \psi_{\text{between-np-ep}}$	✓	✓	30.60	0.90	<u>0.07</u>	<u>0.38</u>	194 278	123	<u>7</u>	<u>4</u>

Comparison to Related Work

To show that our approach generalizes Skavantzios and Link [36], we compare the per-graph metrics on the scenarios supported by both works: S_{No-2}, S_{Nw}, S_{Off-1}, S_{Off-2}, and S_{Off-3}. As Table 10 shows, both approaches yield identical metrics, experimentally corroborating that $\psi_{\text{between-n}}$ is equivalent to the LPG node normalization of [36].

Scenario S_{No-1} could in principle be covered by [36], but is omitted due to a subtle difference in the treatment of keys. Skavantzios and Link [36] treat gUCs as dependencies that determine all property values of a node (S_{No-2}), whereas we treat keys as dependencies that determine a node’s identity, which in turn determines its properties (cf. structurally implied dependencies, Section 4). As discussed in Section 5, such key dependencies do not cause redundancies and require no transformation, yielding a smaller – yet normalized

– graph (visible when comparing S_{No-1} and S_{No-2}). Scenario S_{No-3} is mentioned in [36] but without results, so it is excluded from Table 10.

Discussion

As discussed above, we see that graph-native normalization extends the existing approach to LPG normalization and eliminates redundancies in LPGs. However, it also leads to an increase in the number of graph objects, i.e., nodes and edges, as Figure 7 shows. As an effect, the complexity of a graph’s structure increases as well. While the total number of properties in a graph is reduced as an effect of redundancy reduction, it may happen that the average number of properties in nodes increases, due to reification and the movement of properties, as discussed for S_{Uni-1}. Thus, we conclude

that optimizing for redundancy reduction and a minimal number of graph objects in a graph simultaneously is not possible.

However, with respect to query performance, normalization (and thus redundancy reduction) can lead to improvement. This is an interesting insight compared to the relational data model, where denormalized schemas are sometimes intentionally used to ease and speed up querying of relations.

Another difference between relational and LPGs concerns the definition of dependencies: As relational schemas provide a limited number of possible modeling choices compared to LPGs, the representation dependence of relational FDs and thus relational normalization is less obvious. For LPGs, deciding on the scope of a GO-FD is a more involved task.

8 Conclusion

Building upon related work and in analogy with relational database systems, this paper proposes a graph-native approach to labeled property graph (LPG) normalization. The novel process employs graph transformation rules to normalize graphs into GN-NF and subsequently reduces redundancy. Our evaluation experimentally shows the applicability of our approach using a variety of graphs and functional dependencies; it highlights the tradeoff between having a simple structure and the reduction of redundancies, and it emphasizes the possible impact on query performance.

These observations open several avenues for future research. First, while we assume the availability of GO-FDs, in practice these dependencies may need to be determined automatically, e.g., through mining; however, following related work [36] current mining techniques only can provide heuristics on meaningful dependencies; thus automatic GO-FD discovery is an interesting next step. Another aspect of future work is to consider a broader range of dependencies using complex graph patterns as their scope. Based on our insights on query performance we see the need for further research on rewriting queries for normalized graphs as well. Finally, we plan to investigate to what extent our work can be applied to knowledge graphs in RDF.

References

- [1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.
- [2] Shqiponja Ahmetaj, Iovka Boneva, Jan Hidders, Katja Hose, Maxime Jakubowski, José Emilio Labra Gayo, Wim Martens, Fabio Mogavero, Filip Murlak, Cem Okumus, Axel Polleres, Ognjen Savkovic, Mantas Simkus, and Dominik Tomaszuk. 2025. Common Foundations for SHACL, ShEx, and PG-Schema. In *WWW*. ACM, 8–21.
- [3] Renzo Angles, Angela Bonifati, Stefania Dumbrava, George Fletcher, Alastair Green, Jan Hidders, Bei Li, Leonid Libkin, Victor Marsault, Wim Martens, Filip Murlak, Stefan Plantikow, Ognjen Savkovic, Michael Schmidt, Juan Sequeda, Slawek Staworko, Dominik Tomaszuk, Hannes Voigt, Domagoj Vrgoc, Mingxi Wu, and Dusan Zivkovic. 2023. PG-Schema: Schemas for Property Graphs. *Proc. ACM Manag. Data* 1, 2 (2023), 198:1–198:25.
- [4] Renzo Angles, Angela Bonifati, Stefania Dumbrava, George Fletcher, Keith W. Hare, Jan Hidders, Victor E. Lee, Bei Li, Leonid Libkin, Wim Martens, Filip Murlak, Josh Perryman, Ognjen Savkovic, Michael Schmidt, Juan F. Sequeda, Slawek Staworko, and Dominik Tomaszuk. 2021. PG-Keys: Keys for Property Graphs. In *SIGMOD Conference*. ACM, 2423–2436.
- [5] William Ward Armstrong. 1974. Dependency Structures of Data Base Relationships. In *IFIP Congress*. North-Holland, 580–583.
- [6] Carlo Batini and Monica Scannapieco. 2016. *Data and Information Quality - Dimensions, Principles and Techniques*. Springer.
- [7] Angela Bonifati. 2025. Versatile Property Graph Transformations. *Proc. VLDB Endow.* 18, 12 (2025), 5516–5526.
- [8] Angela Bonifati, George H. L. Fletcher, Hannes Voigt, and Nikolay Yakovets. 2018. *Querying Graphs*. Morgan & Claypool Publishers.
- [9] Antoon Bronsele. 2021. Data Quality Management: An Overview of Methods and Challenges. In *FQAS (Lecture Notes in Computer Science, Vol. 12871)*. Springer, 127–141.
- [10] E. F. Codd. 1970. A Relational Model of Data for Large Shared Data Banks. *Commun. ACM* 13, 6 (1970), 377–387.
- [11] E. F. Codd. 1971. Further Normalization of the Data Base Relational Model. *Research Report / RJ / IBM / San Jose, California* RJ909 (1971).
- [12] E. F. Codd. 1974. Recent Investigations in Relational Data Base Systems. In *IFIP Congress*. North-Holland, 1017–1021.
- [13] Claude Delobel and Richard G. Casey. 1973. Decomposition of a Data Base and the Theory of Boolean Switching Functions. *IBM J. Res. Dev.* 17, 5 (1973), 374–386.
- [14] Lisa Ehrlinger and Wolfram Wöß. 2018. A Novel Data Quality Metric for Minimality. In *QUAT@WISE (Lecture Notes in Computer Science, Vol. 11235)*. Springer, 1–15.
- [15] Nadime Francis, Amélie Gheerbrant, Paolo Guagliardo, Leonid Libkin, Victor Marsault, Wim Martens, Filip Murlak, Liat Peterfreund, Alexandra Rogova, and Domagoj Vrgoc. 2023. GPC: A Pattern Calculus for Property Graphs. In *PODS*. ACM, 241–250.
- [16] Nadime Francis, Amélie Gheerbrant, Paolo Guagliardo, Leonid Libkin, Victor Marsault, Wim Martens, Filip Murlak, Liat Peterfreund, Alexandra Rogova, and Domagoj Vrgoc. 2023. A Researcher's Digest of GQL (Invited Talk). In *ICDT (LIPIcs, Vol. 255)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 1:1–1:22.
- [17] Jelle Hellings, Marc Gyssens, Jan Paredaens, and Yuqing Wu. 2014. Implication and Axiomatization of Functional Constraints on Patterns with an Application to the RDF Data Model. In *FoKS (Lecture Notes in Computer Science, Vol. 8367)*. Springer, 250–269.
- [18] Andreas Heuer, Gunter Saake, and Kai-Uwe Sattler. 2018. *Datenbanken - Konzepte und Sprachen, 6. Auflage*. MITP.
- [19] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan F. Sequeda, Steffen Staab, and Antoine Zimmermann. 2022. Knowledge Graphs. *ACM Comput. Surv.* 54, 4 (2022), 71:1–71:37.
- [20] ISO GQL. ISO/IEC 39075:2024 - Information technology — Database languages — GQL. <https://www.iso.org/standard/76120.html>
- [21] Ernest Lavrinovics, Russa Biswas, Johannes Bjerva, and Katja Hose. 2025. Knowledge Graphs, Large Language Models, and Hallucinations: An NLP Perspective. *J. Web Semant.* 85 (2025), 100844.
- [22] Ulf Leser and Felix Naumann. 2007. *Informationsintegration - Architekturen und Methoden zur Integration verteilter und heterogener Datenquellen*. dpunkt.verlag.
- [23] Junhong Lin, Xiaojie Guo, Yada Zhu, Samuel Mitchell, Erik Altman, and Julian Shun. 2024. FraudGT: A Simple, Effective, and Efficient Graph Transformer for Financial Fraud Detection. In *ICAF*. ACM, 292–300.
- [24] Maude Manouvrier and Khalid Belhajjame. 2026. Graph functional dependencies: Analysis and translation to PG-schema. *Inf. Syst.* 136 (2026), 102633.
- [25] Memgraph. Graph data model. <https://memgraph.com/docs/data-modeling/graph-data-model>
- [26] Neo4j. Graph database concepts. <https://neo4j.com/docs/getting-started/appendix/graphdb-concepts/>
- [27] Neo4j. 2026. *Northwind Graph Example*. <https://github.com/neo4j-graph-examples/northwind>
- [28] International Consortium of Investigative Journalists. Offshore Leaks Database. <https://github.com/ICIJ/offshoreleaks-data-packages>
- [29] Jan Paredaens, Paul De Bra, Marc Gyssens, and Dirk Van Gucht. 1989. *The Structure of the Relational Database Model*. EATCS Monographs on Theoretical Computer Science, Vol. 17. Springer.
- [30] Terence Parr. ANTLR. <https://www.antlr.org>
- [31] Kashif Rabbani, Matteo Lissandrini, Angela Bonifati, and Katja Hose. 2024. Transforming RDF Graphs to Property Graphs using Standardized Schemas. *Proc. ACM Manag. Data* 2, 6 (2024), 242:1–242:25.
- [32] Sherif Sakr, Angela Bonifati, Hannes Voigt, Alexandru Iosup, Khaled Ammar, Renzo Angles, Walid G. Aref, Marcelo Arenas, Maciej Besta, Peter A. Boncz, Khuzaima Daudjee, Emanuele Della Valle, Stefania Dumbrava, Olaf Hartig, Bernhard Haslhofer, Tim Hegeman, Jan Hidders, Katja Hose, Adriana Iamnitci, Vasiliki Kalavri, Hugo Kapp, Wim Martens, M. Tamer Özsu, Eric Peukert, Stefan Plantikow, Mohamed Ragab, Matei Ripeanu, Semih Salihoglu, Christian Schulz, Petra Selmer, Juan F. Sequeda, Joshua Shinavier, Gábor Szárnyas, Riccardo Tommasini, Antonino Tumeo, Alexandru Uta, Ana Lucia Varbanescu, Hsiang-Yun Wu, Nikolay Yakovets, Da Yan, and Eiko Yoneki. 2021. The future is big graphs: a community view on graph processing systems. *Commun. ACM* 64, 9 (2021), 62–71.
- [33] Johannes Schrott. 2026. *London Public Transport*. doi:10.5281/zenodo.18479731
- [34] Johannes Schrott. 2026. *Train Services*. doi:10.5281/zenodo.18478421

- [35] Philipp Skavantzios and Sebastian Link. 2023. Normalizing Property Graphs. *Proc. VLDB Endow.* 16, 11 (2023), 3031–3043.
- [36] Philipp Skavantzios and Sebastian Link. 2025. Third and Boyce-Codd normal form for property graphs. *VLDB J.* 34, 2 (2025), 23.
- [37] Philipp Skavantzios, Kaiqi Zhao, and Sebastian Link. 2021. Uniqueness Constraints on Property Graphs. In *CAiSE (Lecture Notes in Computer Science, Vol. 12751)*. Springer, 280–295.
- [38] Bernhard Thalheim. 2000. *Entity-relationship modeling - foundations of database technology*. Springer.
- [39] Bernhard Thalheim. 2020. Schema Optimisation Instead of (Local) Normalisation. In *FoIKS (Lecture Notes in Computer Science, Vol. 12012)*. Springer, 281–300.
- [40] Domagoj Vrgoc, Carlos Rojas, Renzo Angles, Marcelo Arenas, Diego Arroyuelo, Carlos Buil-Aranda, Aidan Hogan, Gonzalo Navarro, Cristian Riveros, and Juan Romero. 2023. MillenniumDB: An Open-Source Graph Database System. *Data Intell.* 5, 3 (2023), 560–610.

A Appendix

As supplementary material to the experimental evaluation of the graph-native approach to LPG normalization, we provide (1) the minimal covers of the dependencies of each considered scenario, as well as (2) the queries used for the query performance experiment in this appendix.

A.1 Minimal Covers

The minimal covers of the dependencies of each scenario are listed below and are given using the syntax defined for the Python implementation.

- S_{Lon} :
 - $(s:Station:zone\&zoneOriginal)::s.zoneOriginal=>s.zone$
 - $(s:Station:latitude\&longitude)::s.latitude,s.longitude=>s$
 - $(s:Station)-[c:CONNECTED_THROUGH:line\&color\&type]->()::c.line=>c.color,c.type$
- S_{Nw} :
 - $(o:Order:orderId)::o.orderID=>o$
 - $(o:Order:orderId\&orderDate\&customerID\&shipCity\&shipPostalCode\&shipCountry\&shipAddress\&shipRegion)::o.customerID=>o.shipCity,o.shipPostalCode,o.shipCountry,o.shipAddress,o.shipRegion$
- S_{No-1} :
 - $(e:Event:company\&name\&time\&venue)::e.company,e.time=>e$
 - $(e:Event:company\&name\&time\&venue)::e.name=>e.company$
 - $(e:Event:company\&name\&time\&venue)::e.name,e.time=>e.venue$
 - $(e:Event:company\&name\&time\&venue)::e.time,e.venue=>e.name$
- S_{No-2} :
 - $(e:Event:company\&name\&time\&venue)::e.company,e.time=>e.venue$
 - $(e:Event:company\&name\&time\&venue)::e.name=>e.company$
 - $(e:Event:company\&name\&time\&venue)::e.name,e.time=>e.venue$
 - $(e:Event:company\&name\&time\&venue)::e.time,e.venue=>e.name$
- S_{No-3} :
 - $(e:Event\&Confirmed:company\&name\&time\&venue)::e.company,e.time=>e$
 - $(e:Event\&Confirmed:company\&name\&time\&venue)::e.name=>e.company$
 - $(e:Event\&Confirmed:company\&name\&time\&venue)::e.name,e.time=>e.venue$
 - $(e:Event\&Confirmed:company\&name\&time\&venue)::e.time,e.venue=>e.name$
 - $(e:Event\&Confirmed:company\&name\&time\&venue)::e.name,e.company=>e.time$
- S_{Off-1} :
 - $(e:Entity:jurisdiction_description\&countries\&service_provider\&country_codes)::e.countries=>e.country_codes$
 - $(e:Entity:jurisdiction_description\&countries\&service_provider\&country_codes)::e.country_codes=>e.countries$
- S_{Off-2} :
 - $(e:Entity:jurisdiction_description\&valid_until\&countries\&sourceID\&country_codes)::e.countries,e.jurisdiction_description=>e.country_codes$
 - $(e:Entity:jurisdiction_description\&valid_until\&countries\&sourceID\&country_codes)::e.countries,e.sourceID=>e.country_codes$
 - $(e:Entity:jurisdiction_description\&valid_until\&countries\&sourceID\&country_codes)::e.countries,e.valid_until=>e.country_codes$
 - $(e:Entity:jurisdiction_description\&valid_until\&countries\&sourceID\&country_codes)::e.country_codes,e.sourceID=>e.countries$
 - $(e:Entity:jurisdiction_description\&valid_until\&countries\&sourceID\&country_codes)::e.country_codes,e.valid_until=>e.countries$
- S_{Off-3} :
 - $(e:Entity:countries\&service_provider\&country_codes\&jurisdiction_description\&sourceID\&valid_until)::e.countries=>e.country_codes$
 - $(e:Entity:countries\&service_provider\&country_codes\&jurisdiction_description\&sourceID\&valid_until)::e.country_codes=>e.countries$
 - $(e:Entity:countries\&service_provider\&country_codes\&jurisdiction_description\&sourceID\&valid_until)::e.service_provider=>e.sourceID$
 - $(e:Entity:countries\&service_provider\&country_codes\&jurisdiction_description\&sourceID\&valid_until)::e.sourceID=>e.valid_until$

- (e:Entity:countries&service_provider&country_codes&jurisdiction_description&sourceID&valid_until)::
e.valid_until=>e.sourceID
- S_{Ts-1} :
 - ()-[t:STOPS_AT:code]->(s:Station:name)::s.name=>t.code
 - (ts:TrainService:date&number&type)::ts.number,ts.date=>ts.type
 - (ts:TrainService:date&number&operator)-[:STOPS_AT]->():ts.number,ts.date=>ts.operator
 - (ts:TrainService:serviceid)-[:STOPS_AT]->():ts.serviceid=>ts
 - ()-[t:STOPS_AT:stopid]->():t.stopid=>t
 - ()-[t:STOPS_AT:departure&stopid]->(s)::t.stopid=>s
- S_{Ts-2} :
 - ()-[t:STOPS_AT:code]->(s:Station:name)::s.name=>t.code
 - ()-[t:STOPS_AT:code]->(s:Station:name)::t.code=>s.name
 - (ts:TrainService:date&number&type)::ts.number,ts.date=>ts.type
 - (ts:TrainService:date&number&operator)-[:STOPS_AT]->():ts.number,ts.date=>ts.operator
 - (ts:TrainService:serviceid)-[:STOPS_AT]->():ts.serviceid=>ts
 - ()-[t:STOPS_AT:stopid]->():t.stopid=>t
 - ()-[t:STOPS_AT:departure&stopid]->(s)::t.stopid=>s
- S_{Uni-1} :
 - (c:Course)<-[t:TEACHES:usingBook]-():c=>t.usingBook
- S_{Uni-2} :
 - (c:Course:title)<-[t:TEACHES:usingBook]-():c.title=>t.usingBook

A.2 Queries

This sections contain the queries used in the experiment on query performance. For each query, we show the original query before normalization and the rewritten queries for the normalized graph.

- S_{Lon}
 - (1) *Original query:*
 - PROFILE **MATCH** ()-[c:CONNECTED_THROUGH]->() **RETURN DISTINCT** c.line, c.color*After normalization:*
 - PROFILE **MATCH** (c:Ccolorclinetype) **RETURN DISTINCT** c.Cline, c.Ccolor
 - PROFILE **MATCH** ()-[[]-(d:CONNECTED_THROUGH)-[]-()), (d)-[]-(c:Ccolorclinetype) **RETURN DISTINCT** c.Cline, c.Ccolor
 - (2) *Original query:*
 - PROFILE **MATCH** (s:Station)-[c:CONNECTED_THROUGH]-() **WHERE** s.name = "Piccadilly Circus" **RETURN DISTINCT** c.line*After normalization:*
 - PROFILE **MATCH** (s:Station)-[]-(c:CONNECTED_THROUGH)-[:CCOLORCLINETYPE]-(c:Ccolorclinetype) **WHERE** s.name = "Piccadilly Circus" **RETURN DISTINCT** c.Cline
 - (3) *Original query:*
 - PROFILE **MATCH** (s:Station) **WHERE** s.zoneOriginal IS NOT NULL **RETURN DISTINCT** s.zone, s.zoneOriginal*After normalization:*
 - PROFILE **MATCH** (s:SzoneszoneOriginal) **WHERE** s.SzoneOriginal IS NOT NULL **RETURN DISTINCT** s.Szone, s.SzoneOriginal
 - PROFILE **MATCH** (s:Station)-[]-(s:SzoneszoneOriginal) **WHERE** s.SzoneOriginal IS NOT NULL **RETURN DISTINCT** s.Szone, s.SzoneOriginal
 - (4) *Original query:*
 - PROFILE **MATCH** (s:Station)-[c:CONNECTED_THROUGH]-(:Station) **WHERE** s.zone = "6" **RETURN DISTINCT** c.line*After normalization:*
 - PROFILE **MATCH** (s:Station)-[]-(c:CONNECTED_THROUGH)-[]-(:Station), (s)-[]-(z:SzoneszoneOriginal), (c)-[]-(d:Ccolorclinetype) **WHERE** z.Szone = "6" **RETURN DISTINCT** d.Cline
 - (5) *Original query:*
 - PROFILE **MATCH** (s:Station)-[c:CONNECTED_THROUGH]->(t:Station) **WHERE** s.name = "Piccadilly Circus" AND c.line = "Bakerloo" **RETURN DISTINCT** s.name, t.name*After normalization:*
 - PROFILE **MATCH** (s:Station)-[]->(c:CONNECTED_THROUGH)-[]->(t:Station), (c)-[]-(d:Ccolorclinetype) **WHERE** s.name = "Piccadilly Circus" and d.Cline ="Bakerloo" **RETURN** t.name

(6) *Original query:*

– PROFILE MATCH ()-[c:CONNECTED_THROUGH]->() WHERE c.line = "Bakerloo" SET c.color = "Rainbow"

After normalization:

– PROFILE MATCH (c:Ccolorclinectype) WHERE c.Cline = "Bakerloo" SET c.Ccolor = "Rainbow"

– PROFILE MATCH (:CONNECTED_THROUGH)-->(c:Ccolorclinectype) WHERE c.Cline = "Bakerloo" SET c.Ccolor = "Rainbow"

– PROFILE MATCH (:CONNECTED_THROUGH)-->(c:Ccolorclinectype) WHERE c.Cline = "Bakerloo" WITH DISTINCT c SET c.Ccolor = "Rainbow"

• S_{Nw} (1) *Original query:*

– PROFILE MATCH (o:Order) RETURN DISTINCT o.orderID, o.customerID

After normalization:

– PROFILE MATCH (o:Order)-[]-(c:OcustomerIDoshipAddressoshipCityoshipCountryoshipPostalCodeoshipRegion) RETURN DISTINCT o.orderID, c.OcustomerID

(2) *Original query:*

– PROFILE MATCH (o:Order) WHERE o.customerID IS NOT NULL AND o.shipCity IS NOT NULL AND o.shipPostalCode IS NOT NULL AND o.shipCountry IS NOT NULL AND o.shipAddress IS NOT NULL AND o.shipRegion IS NOT NULL RETURN DISTINCT o.customerID

After normalization:

– PROFILE MATCH (o:OcustomerIDoshipAddressoshipCityoshipCountryoshipPostalCodeoshipRegion) WHERE o.OcustomerID IS NOT NULL AND o.OshipCity IS NOT NULL AND o.OshipPostalCode IS NOT NULL AND o.OshipCountry IS NOT NULL AND o.OshipAddress IS NOT NULL AND o.OshipRegion IS NOT NULL RETURN DISTINCT o.OcustomerID

– PROFILE MATCH (:Order)-->(o:OcustomerIDoshipAddressoshipCityoshipCountryoshipPostalCodeoshipRegion) WHERE o.OcustomerID IS NOT NULL AND o.OshipCity IS NOT NULL AND o.OshipPostalCode IS NOT NULL AND o.OshipCountry IS NOT NULL AND o.OshipAddress IS NOT NULL AND o.OshipRegion IS NOT NULL RETURN DISTINCT o.OcustomerID

(3) *Original query:*

– PROFILE MATCH (o:Order) WHERE o.customerID = "FOLKO" SET o.shipCountry = "Rainbow"

After normalization:

– PROFILE MATCH (o:Order)-[]-(c:OcustomerIDoshipAddressoshipCityoshipCountryoshipPostalCodeoshipRegion) WHERE c.OcustomerID = "FOLKO" SET c.OshipCountry = "Rainbow"

– PROFILE MATCH (o:Order)-[]-(c:OcustomerIDoshipAddressoshipCityoshipCountryoshipPostalCodeoshipRegion) WHERE c.OcustomerID = "FOLKO" WITH DISTINCT c SET c.OshipCountry = "Rainbow"

– PROFILE MATCH (c:OcustomerIDoshipAddressoshipCityoshipCountryoshipPostalCodeoshipRegion) WHERE c.OcustomerID = "FOLKO" SET c.OshipCountry = "Rainbow"

• S_{off-1} (1) *Original query:*

– PROFILE MATCH (e:Entity) WHERE e.countries IS NOT NULL AND e.jurisdiction_description IS NOT NULL AND e.country_codes IS NOT NULL RETURN DISTINCT e.countries, e.country_codes

After normalization:

– PROFILE MATCH (c:EcountriesecountryCodes) WHERE c.Ecountries IS NOT NULL AND c.EcountryCodes IS NOT NULL RETURN DISTINCT c.Ecountries, c.EcountryCodes

– PROFILE MATCH (:Entity)-[]-(c:EcountriesecountryCodes) WHERE c.Ecountries IS NOT NULL AND c.EcountryCodes IS NOT NULL RETURN DISTINCT c.Ecountries, c.EcountryCodes

(2) *Original query:*

– PROFILE MATCH (e:Entity) WHERE e.countries IS NOT NULL AND e.service_provider IS NOT NULL AND e.country_codes IS NOT NULL RETURN DISTINCT e.countries, e.service_provider, e.country_codes

After normalization:

– PROFILE MATCH (e:Entity)-[]-(c:EcountriesecountryCodes) WHERE c.Ecountries IS NOT NULL AND c.EcountryCodes IS NOT NULL RETURN DISTINCT c.Ecountries, c.EcountryCodes, e.service_provider

(3) *Original query:*

– PROFILE MATCH (e:Entity) WHERE e.countries IS NOT NULL AND e.jurisdiction_description IS NOT NULL AND e.country_codes IS NOT NULL AND e.countries = "Singapore" SET e.country_codes = "Sp"

After normalization:

– PROFILE MATCH (c:EcountriesecountryCodes) WHERE c.Ecountries IS NOT NULL AND c.EcountryCodes IS NOT NULL AND c.Ecountries = "Singapore" SET c.EcountryCodes = "Sp"

- PROFILE MATCH (:Entity)-[]->(c:EcountriesecountryCodes) WHERE c.Ecountries IS NOT NULL AND c.EcountryCodes IS NOT NULL AND c.Ecountries = "Singapore" SET c.EcountryCodes = "Sp"
- PROFILE MATCH (:Entity)-[]->(c:EcountriesecountryCodes) WHERE c.Ecountries IS NOT NULL AND c.EcountryCodes IS NOT NULL AND c.Ecountries = "Singapore" WITH DISTINCT c SET c.EcountryCodes = "Sp"

- S_{Ts-2}

- (1) Original query:

- PROFILE MATCH ()-[t:STOPS_AT]->(s:Station) WHERE s.name IS NOT NULL AND t.code IS NOT NULL RETURN DISTINCT t.code, s.name

After normalization:

- PROFILE MATCH (n:Snamecode) RETURN DISTINCT n.Sname, n.Tcode
- PROFILE MATCH (s:Station)-->(n:Snamecode), (t:STOPS_AT)-->(n), (t)-->(s) RETURN DISTINCT n.Sname, n.Tcode

- (2) Original query:

- PROFILE MATCH (ts:TrainService)-[t:STOPS_AT]->(s:Station) WHERE ts.type CONTAINS "Nightjet" RETURN DISTINCT ts.number, ts.type, t.code

After normalization:

- PROFILE MATCH (ts:TrainService)-->(t:STOPS_AT)-->(s:Station), (ts)-->(d:Tsdatsnumbersttype), (s)-->(n:Snamecode), (t)-->(n) WHERE d.Tsttype CONTAINS "Nightjet" RETURN DISTINCT d.Tsnumber, d.Tsttype, n.Tcode

- (3) Original query:

- PROFILE MATCH (ts:TrainService) WHERE ts.type CONTAINS "Nightjet" RETURN DISTINCT ts.number, ts.completely_cancelled

After normalization:

- PROFILE MATCH (ts:TrainService)-->(d:Tsdatsnumbersttype) WHERE d.Tsttype CONTAINS "Nightjet" RETURN DISTINCT d.Tsnumber, ts.completely_cancelled

- (4) Original query:

- PROFILE MATCH (ts:TrainService) WHERE ts.type CONTAINS "Sprinter" RETURN DISTINCT ts.number, ts.operator

After normalization:

- PROFILE MATCH (ts:TrainService)-->(d:Tsdatsnumbersttype), (ts)-->(e:Tsdatsnumberstoperator) WHERE d.Tsttype CONTAINS "Sprinter" RETURN DISTINCT d.Tsnumber, e.Tsoperator
- PROFILE MATCH (ts:TrainService)-->(d:Tsdatsnumbersttype), (ts)-->(e:Tsdatsnumberstoperator) WHERE d.Tsttype CONTAINS "Sprinter" RETURN DISTINCT d.Tsnumber, e.Tsoperator

- (5) Original query:

- PROFILE MATCH ()-[t:STOPS_AT]->(s:Station) WHERE s.name IS NOT NULL AND t.code IS NOT NULL AND s.name = "Innsbruck Hbf" SET t.code = "Rainbow"

After normalization:

- PROFILE MATCH (n:Snamecode) WHERE n.Sname = "Innsbruck Hbf" SET n.Tcode = "Rainbow"
- PROFILE MATCH (s:Station)-->(n:Snamecode), (t:STOPS_AT)-->(n), (t)-->(s) WHERE n.Sname = "Innsbruck Hbf" SET n.Tcode = "Rainbow"
- PROFILE MATCH (s:Station)-->(n:Snamecode), (t:STOPS_AT)-->(n), (t)-->(s) WHERE n.Sname = "Innsbruck Hbf" WITH DISTINCT n SET n.Tcode = "Rainbow"